Kombinatorická optimalizace

Tomáš Musil

2023

Contents

1 ILP **1**
1.1 2-Partition Problem . . . 1
1.2 SP - Nejkratší cesty . . . 2
1.3 TSP - Problém obchodního cestujícího . . . 2
1.4 Branch and Bound . . . 2
1.5 Kdy LP vrací celočíselné hodnoty . . . 3
1.6 Převod funkcí do ILP . . . 4

2 Nejkratší cesty **5**
2.1 Převody úloh . . . 5
2.2 Cesta a sled . . . 5
2.3 Trojúhelníková nerovnost . . . 5
2.4 Negativní cykly . . . 6
2.5 Bellmanův princip optimality . . . 6
2.6 Dijkstrův algoritmus . . . 7
2.7 Bellman-Fordův algoritmus . . . 8
2.8 DAG . . . 9
2.9 Floydův algoritmus . . . 10

3 Toky **11**
3.1 Maximální tok . . . 11
3.1.1 Ford-Fulkersonův algoritmus . . . 11
3.1.2 Edmonds-Karpův algoritmus . . . 12
3.1.3 Minimální řez . . . 12
3.1.4 Přípustný tok s bilancemi . . . 13
3.2 Dynamický tok . . . 13
3.3 Tok s minimální cenou . . . 14
3.3.1 Převody úloh . . . 14
3.3.2 Cycle Cancelling algoritmus . . . 15
3.4 Více-komoditní tok s minimální cenou . . . 15
3.5 Párování . . . 16
3.5.1 Párování s maximální kardinalitou . . . 16
3.5.2 Párování s maximální kardinalitou v bipartitním grafu . . . 17
3.5.3 Párování s minimální vahou . . . 17
3.5.4 Perfektní párování s minimální vahou . . . 17

4 Problém batohu **19**
4.1 Zlomkový problém batohu . . . 19
4.2 R-aproximační algoritmus . . . 20
4.3 0/1 problém batohu . . . 20
4.3.1 2-aproximační algoritmus . . . 20
4.3.2 Dynamické programování . . . 21
4.3.3 Aproximační schéma . . . 21

5 Problém obchodního cestujícího **23**
5.1 Silně NP hard problém . . . 23
5.2 Neexistence r-aproximačního algoritmu . . . 24
5.3 Metrické TSP . . . 24
5.3.1 Algoritmus nejbližšího souseda . . . 25
5.3.2 Double-tree algoritmus . . . 25
5.3.3 Christofidův algoritmus . . . 26
5.3.4 Heuristika zlepšování cesty - lokální hledání k-OPT . . . 27

6 Rozvrhování **28**
6.1 Terminologie . . . 28
6.2 Rozvrhování na jednom zdroji ($\alpha = 1$) . . . 29
6.2.1 Úlohy minimalizace C_{max} . . . 29
6.2.2 Úlohy minimalizace $\sum w_j \cdot C_j$ . . . 29

6.2.3 Úlohy minimalizace L_{max} . . . 30
6.2.4 Úloha $1|r_j, \tilde{d}_j|C_{max}$ . . . 30
6.2.5 Bratheyho algoritmus pro $1|r_j, \tilde{d}_j|C_{max}$ . . . 32
6.2.6 Branch and Bound s LP pro $1|prec|\sum w_j \cdot C_j$ . . . 32
6.2.7 EDD - Earliest Due Date first . . . 33
6.2.8 Hornův algoritmus pro $1|pmtn, r_j|L_{max}$ . . . 34
6.2.9 Hornův algoritmus pro $1|pmtn, r_j, d_j = \tilde{d}_j|L_{max}$ . . . 34
6.2.10 Chetto, Silly, Bouchentouf algoritmus pro $1|pmtn, prec, r_j, d_j = \tilde{d}_j|L_{max}$ . . . 35
6.3 Rozvrhování na paralelních identických zdrojích ($\alpha = P$) . . . 35
6.3.1 McNaughtonův algoritmus pro $P|pmtn|C_{max}$ . . . 36
6.3.2 LS - aproximační algoritmus pro $P|prec|C_{max}$ . . . 36
6.3.3 LPT - aproximační algoritmus pro $P||C_{max}$ . . . 37
6.3.4 Rothkopfův algoritmus pro $P||C_{max}$ . . . 37
6.3.5 Muntz&Coffmanův úrovňový algoritmus pro $P|pmtn, prec|C_{max}$ 38
6.4 Project Scheduling . . . 38
6.4.1 Temporální omezení . . . 38
6.4.2 Časově indexovaný model pro $PS1|temp|C_{max}$ . . . 39
6.4.3 Relativně seřazený model pro $PS1|temp|C_{max}$ . . . 40
6.4.4 Relativně seřazený model pro $PSm, 1|temp|C_{max}$ . . . 40
6.4.5 Redukce z $1|r_j, \tilde{d}_j|C_{max}$ na $PS1|temp|C_{max}$ . . . 40
6.4.6 Redukce z $PSm, 1|temp|C_{max}$ na $PS1|temp|C_{max}$ . . . 41
6.4.7 Multiprocesorové úkoly . . . 41

7 Constraint programování **42**
7.1 Constraint satisfaction problém . . . 42
7.2 Search and Propagation . . . 42
7.3 Konzistence hran . . . 43
7.4 AC-3 algoritmus . . . 43

1 ILP

Problém zadaný maticí A a vektorem b, cílem je najít takový vektor x tak, aby $A \cdot x \leq b$. Vektor $x \in \mathbb{Z}^n$ (pro LP je $x \in \mathbb{R}^n$). Obal přípustných řešení je vždy konvexní, v tomto obalu jsou přípustné hodnoty celočíselné (pro LP je prostor příp. řešení konvexní). Nestačí pouze zaokrouhlit výsledek z LP, protože můžeme dostat nevalidní nebo neoptimální řešení. V LP bude optimální řešení vždy v jednom z vrcholů, pro ILP je potřeba vyzkoušet všechna možná celočíselná řešení a vybrat z nich to nejoptimálnější. ILP je NP-hard úloha. Mezi jednotlivými funkcemi ILP platí operátor AND.

Proměnná $M \in \mathbb{Z}_0^+$ značí číslo tak velké, aby (pokud není vynásobeno 0 nebo -1) bylo v nerovnosti větší, než druhá strana funkce. V praxi se často používá maximální hodnota daného primitivního typu nebo nejvyšší možná hodnota, kterou M musí "přebít".

1.1 2-Partition Problem

Snažíme se rozdělit množinu prvků na dvě části tak, aby obě byly stejně velké (např. rozdělení hromady bankovek na dvě části tak. aby obě části měly stejnou cenu).

Převádíme úlohu na rozhodovací problém, tedy jestli je možné takové dvě části vytvořit. Máme n bankovek o hodnotách $p_1, ..., p_n$, kde $p_{i \in 1...n} \in \mathbb{Z}^+$ a $n \in \mathbb{Z}^+$. Existuje taková podmnožina $S \subset \{1, ..., n\}$ taková, že $\sum_{i \in S} p_i = \sum_{i \notin S} p_i$? V ILP budeme mít proměnné $x_i \in \{0, 1\}$, které značí, zda $i \in S$ a definujeme:

$$\sum_{i \in 1...n} x_i \cdot p_i = \sum_{i \in 1...n} (1 - x_i) \cdot p_i$$

Pokud bychom povolili dělitelnost, tedy že jedna bankovka se dá rozdělit na více částí, pak stačí definovat $x_i \in \mathbb{R}_0^+$.

Optimalizační úlohu definujeme pomocí proměnné $C_{max} \in \mathbb{R}_0^+$, kterou se snažíme minimalizovat, a pomocí funkcí:

$$\sum_{i \in 1...n} x_i \cdot p_i \leq C_{max}$$
$$\sum_{i \in 1...n} (1 - x_i) \cdot p_i \leq C_{max}$$

1.2 SP - Nejkratší cesty

SP značí nejkratší cestu mezi bodem a a b. SPT (Shortest Path Tree) značí nejkratší cesty z bodu a do všech ostatních vrcholů. Může se stát, že SP neexistuje.

Pracujeme v orientovaném váženém grafu $G = (V, E)$ s maticí sousednosti $c : V \times V \mapsto \mathbb{R}_0^+$ a s dvěma vrcholy $s, t \in V$. Při výpočtu nejkratších cest definujeme proměnné $l_i \in \mathbb{R}_0^+$, které značí vzdálenost vrcholu i od vrcholu s, proměnnou $n = |V|$ a funkce:

$$l_s = 0$$
$$l_j \leq l_i + c_{ij} \text{ for } i \in 1...n, j \in 1...n, i \neq j$$

1.3 TSP - Problém obchodního cestujícího

Hledáme takový Hamiltonovský cyklus (cyklus procházející všemi vrcholy), který má nejmenší váhu.

Pracujeme v orientovaném váženém grafu $G = (V, E)$ s maticí sousednosti $c : V \times V \mapsto \mathbb{R}_0^+$. Definujeme si proměnné $x_{ij} \in \{0, 1\}$, které značí, jestli hrana mezi vrcholy i a j je v cyklu. Minimalizujeme sumu $\sum_{i \in 1...n} \sum_{j \in 1...n} x_{ij} \cdot c_{ij}$, nejprve definujeme funkce pro zajištění, že do každého vrcholu vstoupíme právě jednou a vystoupíme z něj právě jednou:

$$\sum_{i \in 1...n} x_{ij} = 1 \text{ for } j \in 1...n$$
$$\sum_{j \in 1...n} x_{ij} = 1 \text{ for } i \in 1...n$$

Dále musíme zajistit, že se nevytvoří několik menších cyklů, ale jeden velký cyklus. K tomu si vytvoříme proměnné $s_i \in \mathbb{R}_0^+$, které nám budou reprezentovat čas vstupu do vrcholu i a následující funkci:

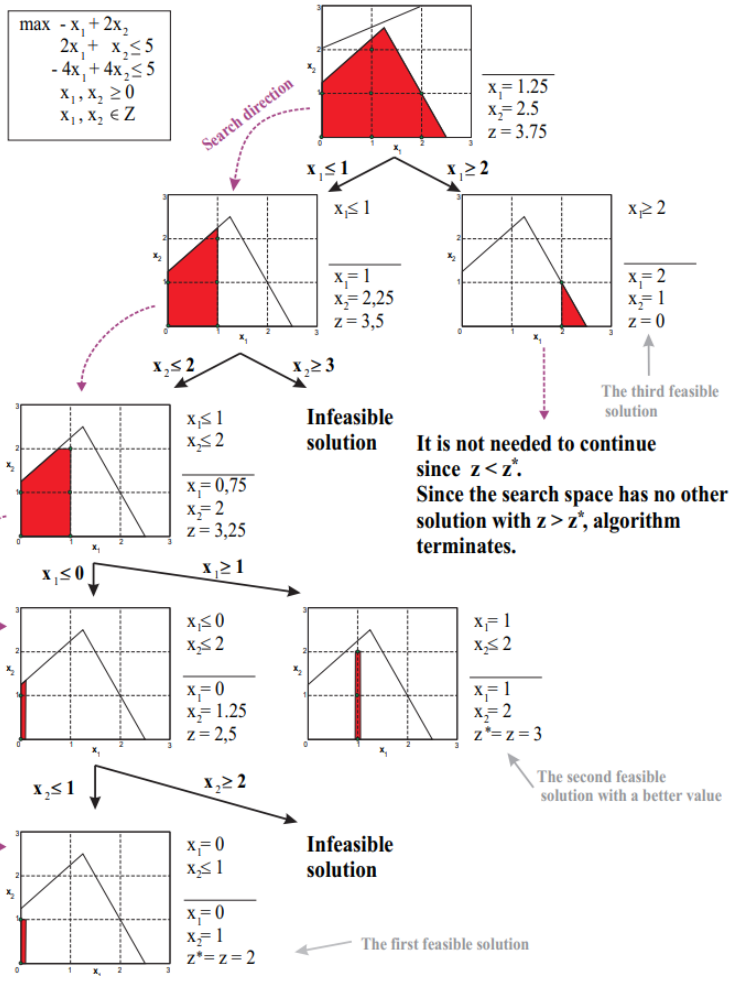
$$s_i + c_{ij} - (1 - x_{ij}) \cdot M \leq s_j \text{ for } i \in 1...n, j \in 2...n$$

1.4 Branch and Bound

Nejprve vyřešíme úlohu jako LP a získáme prostor přípustných řešení P. Pokud najdeme v P extrém, který je celočíselný, máme výsledek. Jinak rozdělíme P podle jedné z os na vybrané hodnotě $k \in \mathbb{Z}$ a dostaneme dva podprostory P_1 a P_2, kdy do P_1 patří hodnoty $\leq k$ a do P_2 patří hodnoty $\geq k+1$. Prostor mezi podprostory P_1 a P_2

nemusím řešit, protože neobsahují žádné celočíselné řešení. Opět hledáme celočíselné extrémy v P_1 a P_2, pokud je najdeme, zapamatujeme si je, jinak rekurzivně dělíme na menší podprostory (můžeme střídat osy, po kterých dělíme), dokud nenajdeme celočíselný extrém, nebo dokud nám nedojdou celočíselné hodnoty v podprostoru. Na závěr vybereme nejoptimálnější celočíselný extrém, který jsme našli.

Některé větve tohoto dělícího stromu můžeme vynechat, jelikož pokud (ne)celočíselný extrém v jedné větvi není optimálnější než již nalezený celočíselný extrém, můžeme tuto větev přerušit, protože nemůže obsahovat optimální řešení.



1.5 Kdy LP vrací celočíselné hodnoty

Pokud bychom pro hledání nejkratších cest měli na vstupu matici incidence místo matice sousednosti, pak by $x_{ij} \in \langle 0, 1 \rangle$ nabývalo vždy hodnot 0 nebo 1, ikdyž jsme

neomezili x_{ij} jako celočíselné. Toto platí, protože matice incidence je tzv. totálně unimodulární a pravé strany všech funkcí jsou celočíselné.

Matice A je totálně unimodulární, pokud se determinanty všech jejích čtvercových podmatic rovnají 0, 1 nebo -1. Obecně, podmínky dostačující pro to, aby matice A o velikosti m/n byla totálně unimodulární jsou: $a_{ij} \in \{0, 1, -1\}$ pro $i = 1...m, j = 1...n$ a každý sloupec v A obsahuje buď právě jednu nenulovou hodnotu nebo jednu 1 a jednu -1.

1.6 Převod funkcí do ILP

$\|x - y\|$	$x - y \leq k$, $y - x \leq k$
$\|x + y\|$	$x + y \leq k$, $-x - y \leq k$
liché číslo	$x = 2k + 1$
AND	$x + y \geq 2$
OR	$x + y \geq 1$
XOR	$x + y = 1$
$x \Rightarrow y$	$x \leq y$
$min(x_i)$	$k \leq x_i$ for $i = 1...n$, maximize k
$max(x_i)$	$k \geq x_i$ for $i = 1...n$, minimize k
jedna z dvou podmínek	$x < 1 + M \cdot (1 - y)$ $1 \leq x + M * y$
k z více podmínek	$x_i < 1 + M \cdot (1 - y_i)$ $\sum_{i \in 1...n} y_i = k$

2 Nejkratší cesty

SP značí nejkratší cestu mezi bodem $s \in V$ a $t \in V$ pro graf $G = (V, E)$ a váhy $c : E \mapsto \mathbb{R}$. Alternativně můžeme počítat SPT (Shortest Path Tree), což je nejkratší cesta od jednoho vrcholu ke všem ostatním, a nebo All Pair Shortest Path, což je nejkratší cesta mezi každými dvěma vrcholy. Cena cesty P je suma vah všech hran v cestě P a značí se $c(E(P))$. Cena nejkratší cesty z s do t se značí $l(s, t)$. Minimální kostra grafu (MST) neodpovídá nejkratším cestám z jednoho vrcholu. Stejně tak Steinerův strom (MST obsahující vybrané vrcholy) je NP hard a není řešitelný přes SP. Hledání nejkratších cest v grafu s negativními cykly je NP hard.

2.1 Převody úloh

Úloha nejkratších cest se dá převést na úlohu nejdelších cest (a naopak) tím, že vynásobíme váhy všech hran -1, pak ale musíme zvolit takový algoritmus, který si se zápornými hranami umí poradit. Pokud jsou vážené nejen hrany, ale i uzly, můžeme každý uzel v nahradit dvojicí uzlů v_1, v_2, mezi kterými bude hrana o váze původního uzlu v, všechny vstupní hrany přesunout do vrcholu v_1 a všechy výstupní do vrcholu v_2 a dále řešit jako klasické SP. Orientovaný graf se dá na neorientovaný graf převést tak, že zduplikujeme všechny hrany a otočíme jejich směr, ale je potřeba dát si pozor na negativní cykly. Pokud jsou váhy hran reprezentované pravděpodobnostmi (např. pravděpodobnost úspěšného doručení balíku), musíme váhy nejprve zlogaritmovat a následně použít algoritmus pro SP.

2.2 Cesta a sled

Sled je jakékoli pořadí navazujících hran (vrcholů) v grafu. Cesta je specifický případ sledu, ve kterém se neopakují hrany ani vrcholy. Pokud v grafu nejsou negativní a nulové cykly, nejkratší sled je také nejkratší cestou. Pokud v grafu nejsou negativní cykly, nejkratší sled obsahuje nejkratší cestu. Jestliže existuje cesta mezi s a t, pak existuje i nejkratší cesta z s do t, což neplatí pro sled, který se v negativním cyklu zacyklí.

2.3 Trojúhelníková nerovnost

Pokud graf neobsahuje negativní cykly, pak pro každou trojici vrcholů i, j, k platí $l(i, j) \leq l(j, k) + l(k, j)$. Definujeme $c(i, j)$ jako váhu hrany z i do j, pak pokud graf

neobsahuje negativní cykly, tak pro sousední vrcholy platí:

$$l(i,j) \leq c(i,j)$$
$$l(i,j) \leq l(i,k) + c(k,j)$$
$$l(i,j) \leq c(i,k) + l(k,j)$$

2.4 Negativní cykly

Hledání nejkratší cesty v grafu s negativními cykly je NP hard. Důkaz pomocí redukce EHC (existence Hamiltonovského cyklu) na SP:

1. Zkopírujeme graf G do grafu G' a všem hranám v G' nastavíme váhu -1.
2. Vytvoříme nový vrchol $v' \in G'$ jako kopii vybraného vrcholu $v \in G'$ včetně jeho hran.
3. Vytvoříme zdrojový vrchol s a cílový vrchol t a hrany (s, v) a (v', t) s váhami 0.
4. Délka nejkratší cesty z s do t v grafu G' se rovná $-|V(G)|$ právě pokud v grafu G existuje Hamiltonovský cyklus.

Jelikož všechny hrany v G' mají váhu -1, nejkratší trasa v tomto grafu je taková, která jde přes co nejvíce hran, tedy přes všechny vrcholy, což odpovídá Hamiltonovské cestě. Hamiltonovský cyklus je složený z Hamiltonovské cesty a hrany mezi vstupním a výstupním vrcholem.

2.5 Bellmanův princip optimality

Máme orientovaný graf $G = (V, E)$ a váhy hran $c : E \mapsto \mathbb{R}$ bez negativních cyklů, cesta P^k je nejkratší cesta mezi vrcholy s a w s maximálně k hranami a hrana $e = (v, w)$ je poslední hrana této cesty. Cesta P^{k-1} (P^k bez poslední hrany e) je nejkratší cestou z vrcholu s do vrcholu v s maximálně $k-1$ hranami. Důkaz sporem:

1. Máme cestu Q_1 z vrcholu s do v, která je kratší než P^{k-1} a **neobsahuje** vrchol w, pak:

   $$c(E(Q_1)) < c(E(P^{k-1}))$$
   $$c(E(Q_1)) + c(e) < c(E(P^{k-1})) + c(e) = c(E(P_k))$$

   Spor, cesta z bodu s do w obsahující Q_1 a hranu e nemůže být kratší než P^k, protože P^k je nejkratší cesta z s do w.

2. Máme cestu Q_2 z vrcholu s do v, která je kratší než P^{k-1} a **obsahuje** vrchol w, pak:

$$c(E(Q_2)) < c(E(P^{k-1}))$$
$$c(E(Q_2[s,w])) + c(E(Q_2[w,v]) < c(E(P^{k-1}))$$
$$c(E(Q_2[s,w])) + \underbrace{c(E(Q_2[w,v]) + c(e)}_{\text{nezáporný cyklus}} < c(E(P^{k-1})) + c(e) = c(E(P_k))$$
$$c(E(Q_2[s,w])) < c(E(P_k))$$

Spor, cesta Q_2 z bodu s do bodu w nemůže být kratší než P^k, protože P^k je nejkratší cesta z s do w.

Bellmanova rovnice: $l(s,w) = min_{w \neq v}\{l(s,v)+c(v,w)\}$. To znamená, že nejkratší cesta je složená z nejkratších podcest.

2.6 Dijkstrův algoritmus

Dijkstrův algoritmus hledá nejkratší cesty z jednoho vrcholu do všech ostatních vrcholů v grafu **bez negativních hran** s asymptotickou složitostí $O(n^2)$ (při použití prioritní fronty $O(m + n \cdot log(n))$. Vstupem je orientovaný graf $G = (V, E)$, váhy hran $c : E \mapsto \mathbb{R}_0^+$ a vrchol $s \in V$. Na výstupu jsou dva vektory l a p, kdy l_v je délka nejkratší cesty z vrcholu s do vrcholu v a p_v je předchůdce v nejkratší cestě z s do v. Pokud je vrchol v nedosažitelný z s, pak $l_v = \infty$ a $p_v = NIL$.

Pseudokód:

$l_s \coloneqq 0$; $l_v \coloneqq \infty$ for $v \neq s$; $R \coloneqq \emptyset$;
while $R \neq V$ **do**
  Find $v \in V \setminus R$ such that $l_v = min_{i \in V \setminus R}(l_i)$;
  $R \coloneqq R \cup \{v\}$; `/* in the first run we add` s `*/`
  `/* calculate` l_w `for every` w `on border of` R `*/`
  foreach $w \in V \setminus R$ *such that* $(v,w) \in E$ **do**
    if $l_w > l_v + c(v,w)$ **then**
      $l_w \coloneqq l_v + c(v,w)$; $p_w \coloneqq v$;
    end
  end
end

Důkaz pravdivosti Dijkstrova algoritmu pomocí indukce: Pro $|R| = 1$ platí $l_s = 0$. což je optimální. Pokud všechny vrcholy v R už mají optimální hodnotu, tak nejlepší možný kandidát v na přidání do R bude mít také optimální hodnotu, jelikož platí

Bellmanova rovnice $l(s,v) = min_{k \neq v}\{(l(s,k) + c(k,v)\}$ pro $k \in R$ i $k \notin R$, jelikož $k \notin R$ nemůže zlepšit hodnotu $l(s,v)$. Všechny cesty v R už jsou optimální, proto optimální $l(s,v)$ půjde vždy přes $k \in R$.

Pro zrychlení Dijkstrova algoritmu můžeme přidat dodatečné informace, jako třeba koncový vrchol (pokud nás zajímá nejkratší cesta do jednoho vrcholu) nebo přidat dodatečnou váhu hranám, které směřují "od" koncového vrcholu. Toto je myšlenka A^* algoritmu. Dále lze využít oboustrannou verzi Dijkstrova algoritmu, která hledá zároveň z vrcholu s a vrcholu t (v opačném směru hran) a jakmile se optimální prostory setkají, ukončí hledání.

2.7 Bellman-Fordův algoritmus

Bellman-Fordův algoritmus hledá nejkratší cesty z jednoho vrcholu do všech ostatních vrcholů v grafu **bez negativních cyklů** s asymptotickou složitostí $O(n \cdot m)$. Vstupem je orientovaný graf $G = (V, E)$, váhy hran $c : E \mapsto \mathbb{R}$ a vrchol $s \in V$. Na výstupu jsou dva vektory l a p, kdy l_v je délka nejkratší cesty z vrcholu s do vrcholu v a p_v je předchůdce v nejkratší cestě z s do v. Pokud je vrchol v nedosažitelný z s, pak $l_v = \infty$ a $p_v = NIL$.

Pseudokód:

$l_s \coloneqq 0$; $l_v \coloneqq \infty$ for $v \neq s$;
for $i \coloneqq 1$ **to** $n - 1$ **do**
  foreach $(v, w) \in E$ **do**
    if $l_w > l_v + c(v, w)$ **then**
      $l_w \coloneqq l_v + c(v, w)$; $p_w \coloneqq v$;
    end
  end
end

Pokud iterace vnějšího cyklu neprovede žádnou změnu v datech, je možné algoritmus ukončit předčasně, při nejhorším skončí algoritmus po $n-1$ iteracích, protože žádná cesta v grafu s n vrcholy nemůže mít více než $n - 1$ hran. Bellman-Fordův algoritmus bere hrany v náhodném pořadí a snaží se vždy zlepšit hodnotu koncového vrcholu pomocí hodnoty počátečního vrcholu a váhy hrany. V nelepším případě jsou hrany seřazené v pořadí nejkratší cesty z vrcholu s do vrcholu t, pak algoritmus může skončit již po první iteraci vnějšího cyklu. V nejhorším případě jsou hrany seřazené v opačném pořadí nejkratší cesty z vrcholu s do vrcholu t, pak algoritmus skončí po k iteracích vnějšího cyklu, kde k je počet hran v nejkratší cestě mezi s a t. Obecně se dá říct, že Bellman-Fordův algoritmus v jedné iteraci přidá do pomyslné nejkratší cesty alespoň jednu hranu, zatímco například Dijkstrův algoritmus přidá v

jedné iteraci právě jednu hranu.

Důkaz pravdivosti Bellman-Fordova algoritmu pomocí indukce: Vezměme l_w^k je hodnota vrcholu w po k iteracích vnějšího cyklu, P^k je nejkratší cesta z s do w o maximálně k hranách a hrana $e = (v, w)$ je poslední hrana této cesty. Pro $k = 1$ platí $l_w^k \leq c(s, w)$, což je optimální. Indukční hypotéza říká, že $l_v^{k-1} \leq c(E(P^{k-1}[s, v]))$ po $k - 1$ iteracích vnějšího cyklu. V k-té iteraci je hrana (v, w) prozkoumána vnitřním cyklem a platí $l_v^k \leq l_v^{k-1} + c(v, w)$. Jelikož P^k obsahuje hranu $(v.w)$ a jelikož platí Bellmanův princip optimality, platí $c(E(P^{k-1}[s, v])) + c(v, w) = c(E(P^k))$.

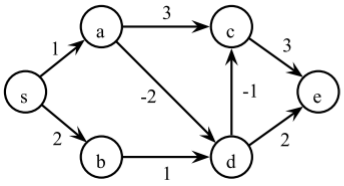
Pak $l_w^k \leq l_v^{k-1} + c(v, w) \leq c(E(P^{k-1}[s, v])) + c(v, w) = c(E(P^k))$, jelikož Bellman-Fordův algoritmus mohl po k iteracích najít lepší cestu s více než k hranami, pokud byly hrany příznivě seřazené. Tento algoritmus je nejoptimálnější známý algoritmus pro úlohu SPT bez negativních cyklů.

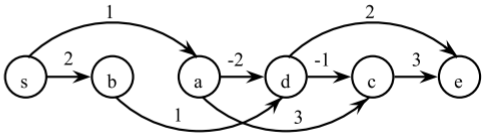
2.8 DAG

DAG (directed acyclic graph) je orientovaný acyklický graf, ve kterém je možné uspořádat vrcholy do jedné řady tak, že všechny hrany směřují stejným směrem. Tímto způsobem se dá reprezentovat například časová osa, resp. jednotlivé časové okamžiky a jejich souvislosti. DAG lze procházet upraveným Bellman-Fordovým algoritmem v asymptotickém čase $O(n + m)$.

Pseudokód:

$l_{v_1} \coloneqq 0$; $l_{v_i} \coloneqq \infty$ for $i \coloneqq 2...n$;
for $i \coloneqq 2$ **to** n **do**
  foreach $(v_j, v_i) \in E$ **do**
    if $l_{v_i} > l_{v_j} + c(v_j, v_i)$ **then**
      $l_{v_i} \coloneqq l_{v_j} + c(v_j, v_i)$; $p_{v_i} \coloneqq v_j$;
    end
  end
end





2.9 Floydův algoritmus

Floydův algoritmus hledá nejkratší cesty mezi **všemi dvojicemi vrcholů** v grafu bez negativních cyklů s asymptotickou složitostí $O(n^3)$. Vstupem je orientovaný graf $G = (V, E)$, váhy hran $c : E \mapsto \mathbb{R}$ a vrchol $s \in V$. Na výstupu jsou dvě matice l a p, kdy l_{ij} je délka nejkratší cesty mezi vrcholy i a j a p_{ij} je předposlední uzel na této cestě. Pokud je vrchol i nedosažitelný z j, pak $l_{ij} = \infty$ a $p_{ij} = NIL$.

Pseudokód:

$l_{ij} \coloneqq c(i, j)$ for $(i, j) \in E$;
$l_{ij} \coloneqq \infty$ for $(i, j) \notin E$ and $i \neq j$;
$l_{ii} \coloneqq 0$ for $i \coloneqq 1...n$;
$p_{ij} \coloneqq i$ for $(i, j) \in E$;
`/* for all` k `check if` l_{ij} `improves */`
for $k \coloneqq 1$ **to** n **do**
  for $i \coloneqq 1$ **to** n **do**
    for $j \coloneqq 1$ **to** n **do**
      if $l_{ij} > l_{ik} + l_{kj}$ **then**
        $l_{ij} \coloneqq l_{ik} + l_{kj}$; $p_{ij} \coloneqq p_{kj}$;
      end
    end
  end
end

Floydův algoritmus umí detekovat záporné cykly. Jelikož v matici l jsou na hlavní ose nejkratší cykly pro daný vrchol, pokud se v buňce l_{ii} objeví záporná hodnota, znamená to, že vrcholem i prochází záporný cyklus.

3 Toky

Síť je definovaná pěticí (G, l, u, s, t), kde $G = (V, E)$ je orientovaný graf, $l : E \mapsto \mathbb{R}_0^+$ a $u : E \mapsto \mathbb{R}_0^+$ značí minimální a maximální kapacitu hran v E, $s \in V$ je zdrojový vrchol a $t \in V$ je koncový vrchol. Tok $f : E \mapsto \mathbb{R}_0^+$ přiřazuje každé hraně e takovou hodnotu, že splňuje 1. Kirchhoffův zákon $\sum_{e \in \delta_v^-} f_e = \sum_{e \in \delta_v^+} f_e$, tedy že suma toků vstupující do vrcholu $v \in V$ se rovná sumě toků vystupujících z vrcholu v, s výjimkou vrcholů s a t. Proveditelný tok je takový tok, pro který platí $f_e \in \langle l_e, u_e \rangle$ pro každé $e \in E$. V dané síti nemusí existovat proveditelný tok, pokud $l_e > 0$.

3.1 Maximální tok

V úloze maximálního toku hledáme takový proveditelný tok f, který maximalizuje $\sum_{e \in \delta_s^+} f_e - \sum_{e \in \delta_s^-} f_e$, tedy maximalizuje tok skrz vrchol s. δ_v^+ je sada hran vycházejících z vrcholu v, δ_v^- je sada hran vcházejících do vrcholu v.

3.1.1 Ford-Fulkersonův algoritmus

Ford-Fulkersonův algoritmus je založený na postupné úpravě toků na hranách podél orientované cesty, zatímco zachovává proveditelnost toku. Dopředná hrana je hrana ve stejném směru, jako cesta z s do t, hrana opačným směrem je zpětná hrana. Zlepšující cesta (augmenting path) pro tok f splňuje $f_e < u_e$ pro všechny dopředné hrany, a $f_e > l_e$ pro všechny zpětné hrany. Dopřednou hranu můžeme brát i jako zpětnou hranu, pokud se po ní vydáme opačným směrem, než je její orientace; to samé platí pro zpětné hrany. Algoritmus pracuje s asymptotickou složiotostí $O(|E|^2 \cdot U)$, kde U je maximální u_e a nalezení zlepšující cesty trvá $O(|E|)$. Výstupem algoritmu je maximální přípustný tok z s do t.

1. Vytvoření proveditelného toku (pokud nejsou všechny $l = 0$, jinak jdeme do kroku 2 s přípustným tokem $f = 0$)
   (a) Přidáme hranu $e = (t, s)$ s $l_e = 0$ a $u_e = \infty$.
   (b) Spočítáme balance vrcholů $v \in V$ jako $b_v = \sum_{e \in \delta_v^+} l_e - \sum_{e \in \delta_v^-} l_e$ tedy rozdíl sumy výstupních l a sumy vstupních l.
   (c) Přidáme vrchol s' a hrany $e = (s', v)$ do všech vrcholů v, pro které platí $b_v > 0$, a nastavíme na nich $l_e = 0$ a $u_e = b_v$.
   (d) Přidáme vrchol t' a hrany $e = (v, t')$ ze všech vrcholů v, pro které platí $b_v < 0$, a nastavíme na nich $l_e = 0$ a $u_e = -b_v$.

(e) Spustíme algoritmus na novém grafu.

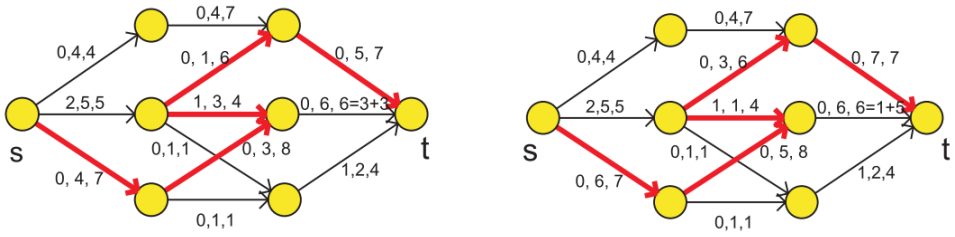
(f) Výsledný tok nastavíme jako přípustný tok do původního grafu.

2. Nalezení maximálního toku

(a) Nalezneme přípustný tok pro všechny hrany $e \in E$ v grafu $G = (V, E)$.

(b) Nalezneme zlepšující cestu P, pokud neexistuje, skončíme.

(c) Nalezeneme minimum z $u_e - f_e$ ($f_e - l_e$ pro zpětné) na hranách $e \in P$, o které pak navýšíme (snížíme pro zpěné) f_e. Vracíme se do kroku b.



Pokud l_e a u_e pro $e \in E$ jsou celočíselné a existuje přípustný tok, pak maximální tok je celočíselný, protože Ford-Fulkersonův algoritmus pracuje pouze s těmito kapacitami a přípustným tokem. Jelikož maximální tok je konečné číslo, algoritmus skončí po konečném počtu kroků. Celočíselnost také vyplývá z totální unimodularity matice incidence, přes kterou je graf definován.

3.1.2 Edmonds-Karpův algoritmus

Algoritmus operuje stejně jako Ford-Fulkersonův algoritmus, ale při výběru zlepšující cesty vybere vždy tu nejkratší (s nejmenším počtem hran). Dá se tedy říci, že Edmonds-Karpův algoritmus pracuje jako Breadth-First Search, zatímco Ford-Fulkersonův pracuje jako Depth-First Search. Edmonds-Karpův algoritmus pracuje s asymptotickou složitostí $O(|E|^2 \cdot |V|)$, což je obecně rychlejší, než Ford-Fulkersonův algoritmus.

3.1.3 Minimální řez

Minimální řez je řez grafem G, obsahující vrchol s, ale nikoli vrchol t. Pro tento řez platí, že hrany které z něj vystupují (do něj vstupují) jsou plně saturované, tedy $f_e = u_e$ pro dopředné hrany a $f_e = l_e$ pro zpětné hrany. Tento řez je výsledkem Ford-Fulkersonova algorimu, když není schopen najít lepší cestu. V jednom grafu může

být více minimálních řezů. Kapacita minimálního řezu A je $\sum_{e\in\delta_A^+} f_e - \sum_{e\in\delta_A^-} f_e$. Hodnota maximálního toku z s do t se rovná kapacitě minimálního řezu A:

$$\sum_{e\in\delta_A^+} f_e - \sum_{e\in\delta_A^-} f_e = \sum_{e\in\delta_A^+} u_e - \sum_{e\in\delta_A^-} l_e$$

3.1.4 Přípustný tok s bilancemi

Máme síť definovanou jako (G, l, u, b) pro $G = (V, E)$, kde $b : V \mapsto \mathbb{R}$, což reprezentuje hodnotu toku, kterou daný vrchol dodává (kladné b_v) nebo spotřebovává (záporné b_v). Takový graf má několik zdrojových vrcholů a několik koncových vrcholů. Hledáme, zda existuje přípustný tok f takový, že:

$$\sum_{e\in\delta_v^+} f_e - \sum_{e\in\delta_v^-} f_e = b_v \text{ pro } v \in V$$

Tuto úlohu můžeme polynomiálně redukovat na úlohu maximálního toku následovně:

1. Přidáme nový vrchol s a hrany (s, v) ke všem zdrojovým vrcholům v s $l_{s,v} = 0$ a $u_{s,v} = b_v$.
2. Přidáme nový vrchol t a hrany (v, t) od všech koncových vrcholů v s $l_{v,t} = 0$ a $u_{v,t} = -b_v$.
3. Zkusíme vyřešit úlohu maximálního toku a pokud tok existuje, má původní úloha odpověď ANO.

3.2 Dynamický tok

Dynamický tok mění svou hodnotu v čase, určuje dobu, za kterou se tok přesune z počátečního vrcholu hrany do koncového vrcholu hrany. Každá hrana $(i, j) \in E$ dostane parametr $d_{ij} \in \mathbb{R}_0^+$, který značí jak dlouho bude trvat se po dané hraně dostat.

Dynamický tok lze převést na statický tok tak, že se pro každou časovou jednotku (např. pro každou hodinu) vytvoří duplicitní sada vrcholů a hrany mezi jednotlivými kopiemi značí posun v čase, tedy hrana $(i, j) \in E$ s $d_{ij} = n$ bude reprezentována hranou $(i_0, j_n) \in E'$. Dále je potřeba vytvořit nové vrcholy s a t tak, že $(s, i_0) \in E'$ a $(i_K, t) \in E'$ pro každé i a maximální čas K.

3.3 Tok s minimální cenou

Máme síť definovanou jako (G, l, u, c, b) pro $G = (V, E)$, kde $c : E \mapsto \mathbb{R}$, což reprezentuje cenu přesunu jedné jednotky toku po dané hraně a b jsou bilance viz výše. Hledáme takový přípustný tok, který minimalizuje $\sum_{e \in E} f_e \cdot c_e$ a splňuje $\sum_{e \in \delta_v^+} f_e - \sum_{e \in \delta_v^-} f_e = b_v$ pro $v \in V$, nebo rozhodneme, že takový tok neexistuje.

3.3.1 Převody úloh

Úlohu maximálního toku můžeme polynomiálně redukovat na úlohu toku s minimální cenou:

1. Přidáme hranu $e = (t, s)$ s $l_e = 0$, $u_e = \infty$ a $c_e = -1$.
2. Nastavíme $c_e = 0$ pro $e \in E \setminus \{(t, s)\}$.
3. Nastavíme $b_v = 0$ pro $v \in V$ (včetně s a t).
4. Cirkulace s minimální cenou (bude negativní) maximalizuje tok na přidané hraně (s, t), tedy maximalizuje tok skrz vrchol s.

Úlohu nejkratší cesty můžeme polynomiálně redukovat na úlohu toku s minimální cenou:

1. Použijeme LP formulaci toku s minimální cenou
2. Nastavíme $b_s = 1$, $b_t = -1$ a $b_v = 0$ pro $v \in V \setminus \{s, t\}$.
3. Nastavíme $l_e = 0$ a $u_e = \infty$ pro $e \in E$.
4. Získáme primární LP formulaci problému nejkratších cest, jelikož můžeme ignorovat kapacity hran a b nám zajistí, že mezi s a t "proběhne" právě jeden tok, reprezentující nejkratší cestu z s do t.

Úlohu Čínského pošťáka (jako TSP, ale musí navštívit každou hranu aspoň jednou) můžeme polynomiálně redukovat na úlohu toku s minimální cenou:

1. Nastavíme $b_v = 0$ pro $v \in V$.
2. Nastavíme $l_e = 1$ a $u_e = \infty$ pro $e \in E$.
3. Vyřešíme úlohu toku s minimální cenou a výsledný tok na každé hraně reprezentuje kolikrát je potřeba danou hranu projít. Spodní omezení nám zaručuje, že každou hranu projdeme aspoň jednou.

3.3.2 Cycle Cancelling algoritmus

Cycle Cancelling algoritmus řeší úlohu toku s minimální cenou v asymptotickém čase $O(|E|^2 \cdot |V| \cdot C \cdot U)$ kde $U = max(u)$ a $C = max(c)$.

1. Nalezneme přípustný tok f s bilancemi v (G, l, u, b).
2. Vytvoříme residuální graf G_f.
   (a) Vytvoříme hrany e' opačné ke všem dopředným hranám e s hodnotou $u_{e'} = u_e - f_e$ a $c_{e'} = c_e$.
   (b) Vytvoříme hrany e' opačné ke všem zpěným hranám e s hodnotou $u_{e'} = f_e - l_e$ a $c_{e'} = -c_e$.
3. Nalezneme cyklus se zápornými hodnotami, pokud neexistuje, končíme.
   (a) Odstraníme hrany e, kde $u_e = 0$.
   (b) Přidáme uzel s a hrany $e = (s, v)$ pro $v \in V(G_f)$ s $c_e = 0$.
   (c) Použijeme (například) Bellman-Fordův algoritmus k nalezení cyklu se zápornými hodnotami.
   (d) Získáme cyklus C se zápornými hodnotami a nebo prohlásíme, že neexistuje.
4. Spočítáme $\gamma = min_{e \in E(C)}(u_e)$ a upravíme tok f na hranách v C o γ. Vrátíme se do kroku 2.

3.4 Více-komoditní tok s minimální cenou

Myšlenkou více-komoditního toku je možnost v jedné síti posílat více různých druhů materiálu, které není možné zaměňovat, ale kolektivně musí splňovat kapacity hran. Máme síť definovanou jako $(G, l, u, c, b^1 ... b^m ... b^{|M|})$ pro $G = (V, E)$, kde $b^m : V \mapsto \mathbb{R}$ značí dodávku/spotřebu komodity m daného vrcholu. Platí $\sum_{v \in V} b_v^m = 0$ pro všechny komodity $m \in M$, tedy že dodávka komodity se rovná její spotřebě. Hledáme takový přípustný tok, který minimalizuje $\sum_{e \in E} \sum_{m \in M} f_e^m \cdot c_e$, tedy přenáší tok s minimální cenou. Přípustný tok splňuje

$$\sum_{e \in \delta_v^+} f_e^m - \sum_{e \in \delta_v^-} f_e^m = b_v^m \text{ pro } v \in V \text{ a } m \in M$$

tedy že 1. Kirchhoffův zákon platí pro každou komoditu $m \in M$. Tato úloha může být vyřešena pomocí LP v polynomiálním čase, ale není zajištěné, že bude existovat celočíselný tok, protože matice incidence není totálně unimodulární (např. když by kapacitou 1 musely protékat dvě komodity zároveň).

3.5 Párování

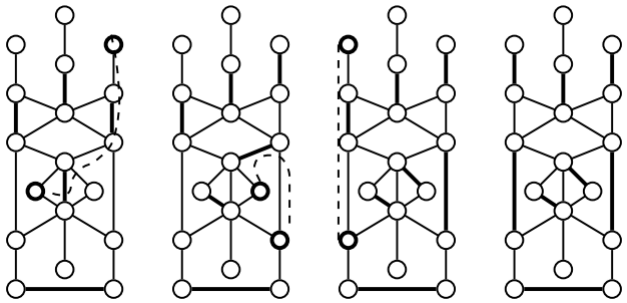
Párování v grafu $G = (V, E)$ je taková množina $P \subseteq E$, pro kterou platí, že žádné dvě hrany v P nesdílí společný vrchol. Když je každý vrchol koncovým vrcholem nějaké hrany v P, jedná se o perfektní párování.

3.5.1 Párování s maximální kardinalitou

Párování s maximální kardinalitou reprezentuje takové párování P, které má největší možný počet hran a lze řešit pomocí hledání M-střídavé cesty.

Mějme párování M v grafu G. Cesta P je M-střídavá cesta, pokud $E(P) \setminus M$ je M-zlepšující a koncové vrcholy nejsou pokryté M. Párování M je maximální právě tehdy, když neexistuje M-zlepšovací cesta. Postup nalezení takové cesty je následující:

1. Najedeme jakékoli párování.
2. Najdeme alternující cestu (cesta na které se střídá nevybraná hrana s vybranou, začíná a končí nevybranou hranou a koncové vrcholy nepatří žádnému párování).
3. Prohodíme vybrané a nevybrané hrany v cestě a jelikož cesta začínala a končila nevybranou hranou, máme jistotu, že výsledné párování bude mít alespoň o jednu hranu víc, než předtím.
4. Pokud jsme našli zlepšující cestu, vrátíme se do kroku 2.



3.5.2 Párování s maximální kardinalitou v bipartitním grafu

Úloha má stejné parametry jako v obecném grafu, ale bipartitní graf nám při řešení usnadní práci. V bipartitním grafu $G = (V, E)$ lze úlohu řešit pomocí redukce na maximální tok následovně:

1. Přidáme vrchol s a hrany (s, v) pro $v \in X$, kde X je jedna ze skupin v bipartitním grafu.
2. Přidáme vrchol t a hrany (v, t) pro $v \in Y$, kde Y je druhá ze skupin v bipartitním grafu.
3. Zkontrolujeme, že hrany vedou od s do X, od X do Y a od Y do t.
4. Nastavíme $u_e = 1$ a $l_e = 0$ na $e \in E$.
5. Vyřešíme úlohu maximálního toku a získáme párování s maximální kardinalitou.

3.5.3 Párování s minimální vahou

Hledáme takové párování s maximální kardinalitou, které minimalizuje cenu na vybraných hranách ve váženém grafu.

3.5.4 Perfektní párování s minimální vahou

Úloze se také říká Assignment Problem. V kompletním váženém bipartitním grafu (každý vrchol ze skupiny X je spojený s každým vrcholem ze skupiny Y), kde $|X| = |Y|$ hledáme perfektní párování, které minimalizuje cenu na vybraných hranách.

Úlohu lze řešit pomocí redukce na tok s minimální cenou podobně, jako u párování s maximální kardinalitou v bipartitním grafu:

1. Přidáme vrchol s a hrany $e = (s, v)$ pro $v \in X$, kde X je jedna ze skupin v bipartitním grafu a nastavíme $c_e = 0$.
2. Přidáme vrchol t a hrany (v, t) pro $v \in Y$, kde Y je druhá ze skupin v bipartitním grafu a nastavíme $c_e = 0$.
3. Zkontrolujeme, že hrany vedou od s do X, od X do Y a od Y do t.
4. Nastavíme $u_e = 1$ a $l_e = 0$ na $e \in E$, váhy hran jsou ceny hran.

5. Vyřešíme úlohu toku s minimální cenou a získáme perfektní párování s minimální vahou

Úlohu lze také řešit pomocí tzv. Maďarského algoritmu, který je jen pro zájemce.

4 Problém batohu

Máme nějaké množství předmětů o určité váze a ceně a dále máme batoh, který má omezenou nosnost. Snažíme se najít takovou kombinaci předmětů, abychom maximalizovali cenu a nepřekročili nosnost batohu. Jedná se o NP hard úlohu, ale existují na ní jednoduché aproximační algoritmy.

4.1 Zlomkový problém batohu

Pokud povolíme dělení jednotlivých předmětů na menší části, vzniká nám nová formulace problému batohu. Máme celá nezáporná čísla $n, c_1...c_n, w_1...w_n, W$, kde n je počet prvků, c_i je cena i-tého prvku, w_i je váha i-tého prvku a W je nosnost batohu. Hledáme taková racionální čísla $x_1...x_n$, že $0 \leq x_i \leq 1$ pro $i = 1...n$, platí $\sum_{i=1}^{n} x_i \cdot w_i \leq W$ a že $\sum_{i=1}^{n} x_i \cdot c_i$ je maximální.

Jelikož předměty mohou být dělené, můžeme tuto úlohu vyřešit v polynomiálním čase:

1. Pokud $\sum_{i=1}^{n} w_i \leq W$, úloha je triviální, protože vezmeme všechny prvky.
2. Seřadíme a přeindexujeme prvky podle jejich relativních cen:
   $\frac{c_1}{w_1} \geq \frac{c_2}{w_2} \geq ... \geq \frac{c_n}{w_n}$.
3. V novém pořadí najdeme první prvek, který se do batohu nevejde:
   $h \coloneqq min(j \in \{1, ..., n\} : \sum_{i=1}^{j} w_i > W)$.
4. Odřízneme takovou část prvku h, aby se vešla do batohu:
   $x_i \coloneqq 1$ pro $j = 1...h-1$, tedy vložíme všechny prvky před h celé
   $x_h \coloneqq \frac{W - \sum_{i=1}^{h-1} w_i}{w_h}$
   $x_i \coloneqq 0$ pro $i = h+1...n$, tedy prvky po h vynecháme

Seřazení prvků trvá $O(n \cdot log(n))$ a výpočet h trvá $O(n)$, tento algoritmus tedy vyřeší zlomkový problém batohu s asymptotickou složitostí $O(n \cdot log(n))$. Existuje ještě rychlejší algoritmus, který řeší úlohu v $O(n)$ čase redukcí na problém váženého mediánu.

4.2 R-aproximační algoritmus

Algoritmus A pro problém J se nazývá r-aproximační, pokud existuje $r \geq 1$ takové, že

$$\overbrace{r \cdot J^*(I) \geq \underbrace{J^A(I)}_{\text{maximalizace}}}^{\text{minimalizace}} \geq \frac{1}{r} \cdot J^*(I)$$

pro všechny instance I problému J, kde $J^*(I)$ je optimální řešení instance I. To znamená, že výsledek aproximačního algoritmu A bude maximálně r-krát horší, než optimální řešení, tedy bude r-krát větší při minimalizaci a r-krát menší při maximalizaci.

4.3 0/1 problém batohu

Máme celá nezáporná čísla $n, c_1...c_n, w_1...w_n, W$, kde n je počet prvků, c_i je cena i-tého prvku, w_i je váha i-tého prvku a W je nosnost batohu. Hedáme takovou podmnožinu $S \subseteq \{1, ..., n\}$, že platí $\sum_{i \in S} w_i \leq W$ a že $\sum_{i \in S} c_i$ je maximální. Název vychází z faktu, že dané číslo v S buď je (1) nebo není (0).

4.3.1 2-aproximační algoritmus

Vyjděme z předpokladu, že seznam prvků je netriviální ($\sum_{i=1}^{n} w_i > W$), je seřazený sestupně podle relativní ceny ($\frac{c_i}{w_i}$) a máme prvek h, který se do batohu nevejde celý. Pak výběr té lepší ze dvou možností $\{1, ..., h-1\}$ a $\{h\}$ je 2-aproximační algoritmus problému batohu s asymptotickou složitostí $O(n)$. Důkaz $r = 2$:

1. Vyřadíme všechny prvky, pro které $w > W$.
2. Pokud $\sum_{i=1}^{n} w_i \leq W$, tak máme optimální řešení, jelikož všechny prvky se vejdou do batohu.
3. Jelikož $\sum_{i=1}^{h} c_i$ je horní omezení optimální hodnoty, lepší ze dvou možných řešení $\sum_{i=1}^{h-1} c_i$ a c_h je alespoň polovina optimální hodnoty:

$$J^A(I) = max\left(\sum_{i=1}^{h-1} c_i, c_h\right) \geq \frac{\sum_{i=1}^{h} c_i}{2} > \frac{1}{2} \cdot J^*(I)$$

4.3.2 Dynamické programování

Jedná se o psaudopolynomiální algoritmus s asymptotickou složitostí $O(n \cdot C)$, kde C je velmi velká konstanta. Proměnná x_k^j reprezentuje minimální možnou váhu s cenou k, která může být získána výběrem prvků z množiny $\{1, ..., j\}$. Předmět i je přidaný do výběru z $\{1, ..., j\}$, pokud pro danou cenu k dosahuje výsledný výběr menší nebo stejné váhy, jako $\{1, ..., j-1\}$. Hodnota x_k^j je počítána následovně:

$$x_k^j := \begin{cases} x_{k-c_j}^{j-1} & \text{pokud byl prvek } j \text{ přidán} \\ x_k^{j-1} & \text{pokud nebyl prvek } j \text{ přidán} \end{cases}$$

V proměnné s_k^j si zapamatujeme který z dvou možných stavů nastal a pomocí ní pak zrekonstruujeme výběr.

Celočíselné ceny

$w = (21, 35, 52, 17)$ $W = 100$
$c = (10, 20, 30, 10)$

	0	10	20	30	40	50	60	70
0	0	∞	∞	∞	∞	∞	∞	∞
1	0	21	∞	∞	∞	∞	∞	∞
2	0	21	35	56	∞	∞	∞	∞
3	0	21	35	52	73	87	~~108~~	∞
4	0	17	35	52	69	87	~~104~~	∞

Výsledek: {2, 3}

Celočíselné hodnoty

$w = (2, 3, 4, 5)$ $W = 8$
$c = (3.1, 4.2, 5.1, 4.3)$

	0	1	2	3	4	5	6	7	8
0	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
1	0	$-\infty$	3,1	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
2	0	$-\infty$	3,1	4,2	$-\infty$	7,3	$-\infty$	$-\infty$	$-\infty$
3	0	$-\infty$	3,1	4,2	5,1	7,3	8,2	9,3	$-\infty$
4	0	$-\infty$	3,1	4,2	5,1	7,3	8,2	9,3	8,5

Výsledek: {2, 3}

Pokud jsme ochotni potenciálně ztratit určitou přesnost výpočtu za získáním větší rychlosti algoritmu, můžeme všechny ceny vydělit nějakou konstantou t, výsledná asymptotická složitost pak bude $O(n\frac{C}{t})$. Pak se z tohoto algoritmu stává t-aproximační algoritmus.

4.3.3 Aproximační schéma

Máme celá nezáporná čísla $n, c_1...c_n, w_1...w_n, W, \epsilon$, kde ϵ značí o kolik procent jsme ochotni vzdálit se od optima. Asymptotická složitost takového algoritmu je $O(n^2 \cdot \frac{1}{\epsilon})$.

Výstupem je taková podmnožina $S \subseteq \{1, ..., n\}$, že platí:

$$\sum_{j \in S} w_j \leq W$$

$$\sum_{j \in S} c_j \geq \frac{1}{1+\epsilon} \cdot \sum_{j \in U} c_j \text{ pro } U \subseteq \{1, ..., n\} \text{ splňující } \sum_{j \in U} w_j \leq W$$

Postup:

1. Spustíme 2-aproximační algoritmus a označíme výsledek jako S_1 s cenou $c_{S_1} = \sum_{j \in S_1} c_j$.
2. Nastavíme $t \coloneqq max(1, \frac{\epsilon \cdot c_{S_1}}{n})$ a $c'_j \coloneqq \lfloor \frac{c_j}{t} \rfloor$ pro $j = 1...n$.
3. Spustíme dynamické programování batohu s cenami c'_j a horním omezením $C = \frac{2c_{S_1}}{t}$ a označíme výsledek jako S_2 s cenou $c_{S_2} = \sum_{j \in S_2} c_j$.
4. Pokud $c_{S_1} > c_{S_2}$, pak $S \coloneqq S_1$, jinak $S \coloneqq S_2$.

5 Problém obchodního cestujícího

V problému existence Hamiltonovské kružnice pracujeme s neorientovaným grafem, ve kterém hledáme takovou kružnici, která navštíví každý vrchol právě jednou; jedná se o NP complete rozhodovací problém. V problému obchodního cestujícího (TSP) pracujeme s neorientovaným váženým grafem, ve kterém hledáme takovou kružnici, která navštíví každý vrchol právě jednou a minimalizuje cenu využitých hran; jedná se o NP hard optimalizační úlohu. Pokud je na vstupu orientovaný vážený graf, pak se jedná o asymetrické TSP. V úplném neorientovaném grafu o n vrcholech je $\frac{(n-1)!}{2}$ Hamiltonovských kružnic.

5.1 Silně NP hard problém

NP hard problémy jsou takové, pro které neumíme najít polynomiální algoritmy, které by je řešili, jsme ale schopni najít pseudopolynomiální algoritmy, které nám úlohy řeší (např. dynamické programování u problému batohu). Pro silně NP hard problémy neumíme najít ani pseudopolynomiální algoritmy na řešení a ani při polynomiálním omezení vstupních parametrů nepřestávají být NP hard.

Mějme optimalizační problém L. Pro polynom p mějme L_p jako omezení takových instancí I problému L, že I se skládá z nezáporných celých čísel s $largest(I) \leq p(size(I))$, tedy že parametry L_p jsou omezeny polynomem p. L je silně NP hard, pokud existuje takový polynom p, že L_p je NP hard. Pokud L je silně NP hard, pak nemůže být vyřešeno pseudopolynomiálním allgoritmem, pokud P $\neq$ NP. Důkaz, že problém obchodního cestujícího je silně NP hard, pomocí redukce z existence Hamiltonovské kružnice:

1. Mějme L = TSP a L_p = TSP s omezením vah na $\{1, 2\}$.
2. Mějme neorientovaný graf $G = (V, E)$, ve kterém chceme nalézt Hamiltonovskou kružnici.
3. Vytvoříme instanci TSP takovou, že každý vrchol z G přiřadíme k nějakému vrcholu v úplném neorientovaném grafu K_n, kde hrana (i, j) má váhu:

$$c_{ij} = \begin{cases} 1 & \text{pokud } (i,j) \in E \\ 2 & \text{pokud } (i,j) \notin E \end{cases}$$

4. G má Hamiltonovskou kružnici právě tehdy, když optimální řešení TSP je rovné $n = |V|$, tedy když TSP vybralo pouze hrany z původního G.

5.2 Neexistence r-aproximačního algoritmu

Předpokládáme, že P $\neq$ NP. Dokážeme neexistenci r-aproximačního algoritmu pro $r \geq 1$ sporem:

1. Předpokládejme, že každá instance Hamiltonovské kružnice se dá polynomiálně redukovat na TSP nepřesně vyřešeno r- aproximačním algoritmem A.
2. Mějme neorientovaný graf $G = (V, E)$, ve kterém chceme nalézt Hamiltonovskou kružnici.
3. Vytvoříme instanci TSP takovou, že každý vrchol z G přiřadíme k nějakému vrcholu v úplném neorientovaném grafu K_n, kde hrana (i, j) má váhu:

$$c_{ij} = \begin{cases} 1 & \text{pokud } (i,j) \in E \\ 2 + (r-1) \cdot n & \text{pokud } (i,j) \notin E \end{cases}$$

4. Použijeme A pro vyřešení instance.
   (a) Pokud je výsledek v intervalu $\langle n, r \cdot n \rangle$, pak existuje Hamiltonovská kružnice
   (b) Jinak je výsledek větší nebo roven $(n-1) + 2 + (r-1) \cdot n = r \cdot n + 1$ a G neobsahuje Hamiltonovskou kružnici
5. Jelikož algoritmus A umí v polynomiálním čase řešit problém existence Hamiltonovské kružnice, platilo by, že P = NP, což je spor.

5.3 Metrické TSP

V metrickém TSP je na vstupu takový úplný vážený neorientovaný graf K_n, že každá trojice vrcholů i, j, k splňuje trojúhelníkovou nerovnost, tedy $c_{ij} + c_{jk} \geq c_{ki}$. Metrické TSP je silně NP hard, jelikož váhy hran 1 a 2 neporušují trojúhelníkovou nerovnost, takže neexistuje pseudopolynomiální algoritmus, který by úlohu řešil. Narozdíl od obecného TSP však existují aproximační algoritmy. Z obecného TSP můžeme udělat metrické TSP tak, že k váze všech hran přičteme konstantu h, ale tento krok nevede k aproximačnímu algoritmu pro obecné TSP, protože přičtení konstanty ke všem hranám není ekvivalentní úprava.

5.3.1 Algoritmus nejbližšího souseda

Jedná se o heuristický algoritmus, nikoli aproximační. Vstupem je instance metrického TSP, tedy graf $K_n = (V, E)$ a ceny hran c. Výstupem je Hamiltonovská kružnice H. Algoritmus pracuje s asymptotickou složitostí $O(n^2)$. Postup, kdy v každém kroku vybíráme nejkratší cestu z posledního vrcholu:

1. Vybereme náhodný vrchol $v_1 \in V$.
2. Vybereme vrchol $v_i \in V \setminus \{v_1, ..., v_{i-1}\}$ tak, aby cena (v_{i-1}, v_i), kde v_{i-1} je poslední vybraný vrchol, byla minimální.
3. Pokud nemáme vybrány všechny vrcholy, vrátíme se do bodu 2.
4. Spojíme poslední vrchol s vrcholem v_1 a získáme Hamiltonovskou kružnici H.

5.3.2 Double-tree algoritmus

Jedná se o 2-aproximační algoritmus. Vstupem je instance metrického TSP, tedy graf $K_n = (V, E)$ a ceny hran c. Výstupem je Hamiltonovská kružnice H. Algoritmus pracuje s asymptotickou složitostí $O(n^2)$. Postup:

1. Najdeme minimální kostru T v grafu K_n.
2. Pro každou hranu vytvoříme hranu opačnou, tím získáme multigraf, ve kterém najdeme Eulerovský tah L.
3. Převedeme Eulerovský tah L do Hamiltonovského cyklu H v úplném grafu K_n.
   (a) Vytvoříme pořadí vrcholů v Eulerovském tahu L.
   (b) Vynecháme ty vrcholy, které už byly jednou navštívené.
   (c) Získáme Hamiltonovskou kružnici H, jelikož graf K_n je úplný, víme jistě, že všechny dvojice po sobě jdoucích vrcholů v H mezi sebou mají hranu.

Důkaz aproximačního faktoru $r = 2$:

1. Jelikož v grafu K_n platí trojúhelníková nerovnost, vynechání duplicitních vrcholů při tvorbě H nemůže zvětšit její cenu oproti L, tedy $c(E(L)) \geq c(E(H))$.
2. Pokud smažeme v kružnici jednu hranu, získáme strom, proto platí $OPT(K_n, c) \geq c(E(T))$, kde $OPT(K_n, c)$ je optimální řešení.
3. Jelikož L vzniklo zdvojením hran v T, platí $2 \cdot c(E(T)) = c(E(L))$.

4. Z toho platí $2 \cdot OPT(K_n, c) \geq 2 \cdot c(E(T)) = c(E(L)) \geq c(E(H))$, tedy že cena H je menší nebo rovna dvojnásobku ceny optimálního řešení.

5.3.3 Christofidův algoritmus

Jedná se o $\frac{3}{2}$-aproximační algoritmus. Vstupem je instance metrického TSP, tedy graf $K_n = (V, E)$ a ceny hran c. Výstupem je Hamiltonovská kružnice H. Algoritmus pracuje s asymptotickou složitostí $O(n^3)$. Postup:

1. Najdeme minimální kostru T v grafu K_n.
2. Vezmeme W jako množinu všech vrcholů v T s lichým stupněm; platí, že $|W|$ je sudé.
3. Najdeme perfektní párování s minimální vahou M mezi vrcholy ve W v grafu K_n, tedy vytvoříme z vrcholů ve W dvojice tak, že suma vybraných hran je minimální; vrcholy ve dvojici mohou mít společnou hranu v T.
4. Spojením T a M vznikde multigraf, ve kterém najdeme Eulerovský tah L.
5. Převedeme Eulerovský tah L na Hamiltonovskou kružnici H v úplném grafu K_n, vynecháním již navštívených vrcholů.

Důkaz aproximačního faktoru $r = \frac{3}{2}$:

1. Jelikož v grafu K_n platí trojúhelníková nerovnost, vynechání duplicitních vrcholů při tvorbě H nemůže zvětšit její cenu oproti L, tedy $c(E(L)) \geq c(E(H))$.
2. Pokud smažeme v kružnici jednu hranu, získáme strom, proto platí $OPT(K_n, c) \geq c(E(T))$, kde $OPT(K_n, c)$ je optimální řešení.
3. Jelikož perfektní párování M vybírá každou druhou hranu v cestě a jelikož vybírá vždy polovinu s menší vahou, platí $\frac{OPT(K_n,c}{2} \geq c(E(M))$.
4. Jelikož L vzniklo spojením T a M, platí $c(E(T)) + c(E(M)) = c(E(L))$.
5. Z toho platí $\frac{3}{2} \cdot OPT(K_n, c) \geq c(E(T)) + c(E(M)) = c(E(L)) \geq c(E(H))$, tedy že cena H je menší nebo rovna třem polovinám ceny optimálního řešení.

5.3.4 Heuristika zlepšování cesty - lokální hledání k-OPT

V praxi se jedná o jednu z nejúspěšnějších technik pro TSP. Popisuje velice jednoduchou techniku, kterou lze využít k řešení jiných optimalizačních problémů. Postup:

1. Najdeme jakoukoli Hamiltonovskou kružnici H pomocí nějaké heuristiky.
2. Opakovaně ji vylepšujeme pomocí "lokálních úprav" (např. smazáním k hran a rekonstrukcí kružnice, to dělá k-OPT algoritmus).

Záleží na dvou hlavních rozhodnutích: jaké úpravy jsou povolené a jestli provedeme danou úpravu (např. můžeme provádět pouze zlepšující úpravy).

6 Rozvrhování

Úkolem rozvrhování je seřazení všech úkolů na jednotlivé procesory tak, aby se dodržely požadavky na čas spuštění, čas ukončení, dobu trvání úkolu a závislosti mezi úkoly. Rozhodovací varianta úlohy nám říká, zda jsou zadané úkoly s podmínkami rozvrhnutelné, optimalizační úloha minimalizuje čas ukončení posledního úkolu nebo délku zpoždění. Každý úkol může být v jednu chvíli zpracováván maximálně jedním procesorem, každý procesor může v jednu chvíli zpracovávat maximálně jeden úkol. Můžeme znát všechny úkoly už před začátkem rozvrhování (off-line) nebo nám mohou úkoly postupně přicházet (on-line).

6.1 Terminologie

T_j úkol s indexem j

s_j start time (čas začátku) úkolu T_j

p_j processing time (doba trvání) úkolu T_j

C_j completion time (čas skončení) úkolu T_j

L_j lateness (zpoždění) úkolu T_j

Pro popis zadání se využívá Grahamova notace $\alpha|\beta|\gamma$

- α popis zdrojů
  - α_1 první část (jedna z níže uvedených)
    - 1 k dispozici je jeden procesor
    - P paralelní zpracování
    - PS project scheduling
  - α_2 druhá část (jedna z níže uvedených)
    - $\emptyset$ k dispozici je libovolné množství procesorů
    - 2 k dispozici jsou dva procesory (může být jakékoli číslo)
    - m, R m typů zdrojů s kapacitami R (project scheduling)
- β popis omezení (více z níže uvedených)
  - r_j úkoly mají release time (nejdřívější možný čas začátku)
  - d_j úkoly mají due date (nejpozdější očekávaný čas skončení), lze ho překročit a pak nastává $L_j > 0$

$\tilde{d}_j$ úkoly mají deadline (nejpozdější možný čas skončení), nelze ho překročit

pmtn úkoly mají preempci, takže je můžeme pozastavit a pokračovat v nich později

prec úkoly mají precedenci, tedy jsou na sobě závislé

temp temporal constraints (úkol může začít až po nějaké době od začátku předchozího)

$set - up$ set-up time (úkoly musí mít mezi sebou časový prostor)

γ popis minimalizovaného kritéria (jedna z níže uvedených)

C_{max} čas skončení posledního úkolu

$\sum C_j$ suma časů dokončení

$\sum C_j \cdot w_j$ suma vážených časů dokončení

L_{max} celkové zpoždění

$\emptyset$ rozhodovací problém (zda lze úkoly vůbec rozvrhnout)

6.2 Rozvrhování na jednom zdroji ($\alpha = 1$)

Naším cílem je najít takové pořadí úkolů, abychom minimalizovali celkovou dobu provádění úkolů.

6.2.1 Úlohy minimalizace C_{max}

$1|prec|C_{max}$ triviální, úkoly jsou seřazené podle jejich precedence, $C_{max} = \sum_{j=1}^{n} p_j$

$1||C_{max}$ triviální, úkoly jsou seřazené náhodně, $C_{max} = \sum_{j=1}^{n} p_j$

$1|r_j|C_{max}$ triviální, úkoly jou seřazené vzestupně podle jejich r_j

$1|\tilde{d}_j|C_{max}$ triviální, úkoly jsou seřazené vzestupně podle jejich $\tilde{d}_j$, může být vyřešeno pomocí EDF - Earliest Deadline First, může se stát, že rozvrh neexistuje

$1|r_j, \tilde{d}_j|C_{max}$ NP hard, lze polynomiálně redukovat z 3-Partititon problému, pro $p_j = 1$ existuje polynomiální algoritmus

6.2.2 Úlohy minimalizace $\sum w_j \cdot C_j$

Tyto úlohy mají smysl převážně když chceme minimalizovat čas čekání každého úkolu, než na něj přijde řada.

$1\|\|\sum C_j$	triviální, úkoly jsou seřazené vzestupně podle jejich p_j (SPT - Shortest Processing Time first)
$1\|\|\sum w_j \cdot C_j$	triviální, úkoly jsou seřazené vzestupně podle $\frac{p_j}{w_j}$ (vážené SPT)
$1\|r_j\|\sum C_j$	NP hard
$1\|pmtn, r_j\|\sum C_j$	může být vyřešeno pomocí upraveného SPT
$1\|pmtn, r_j\|\sum w_j \cdot C_j$	NP hard
$1\|\tilde{d_j}\|\sum C_j$	může být vyřešeno pomocí upraveného SPT
$1\|\tilde{d_j}\|\sum w_j \cdot C_j$	NP hard
$1\|prec\|\sum C_j$	NP hard

6.2.3 Úlohy minimalizace L_{max}

Minimalizace L_{max} automaticky znamená, že existuje d_j, i když není v Grahamově notaci napsané. L_{max} může být záporné, pokud dokončíme všechny úkoly před jejich due date.

$1\|\|L_{max}$	triviální, úkoly jsou seřazené vzestupně podle due date (EDD - Earliest Due Date first)
$1\|r_j\|L_{max}$	NP hard
$1\|r_j, p_j = 1\|L_{max}$	řešíme pomocí iterujícího EDD
$1\|pmtn, r_j\|L_{max}$	řešíme pomocí Hornova algoritmu
$1\|pmtn, r_j, d_j = \tilde{d_j}\|L_{max}$	řešíme pomocí Hornova algoritmu (EDF)
$1\|pmtn, prec, r_j, d_j = \tilde{d_j}\|L_{max}$	řešíme pomocí převodu na nezávislé úkoly a pak Hornovým algoritmem

6.2.4 Úloha $1|r_j, \tilde{d_j}|C_{max}$

Důkaz, že úloha je NP hard, provedeme pomocí redukce z 3-Partititon problému, který je NP complete:

1. Mějme instanci $I_{3P} = (A, B)$ 3-Partition problému
   (a) Multimnožina A obsahuje $3m$ celých čísel $a_1, a_2, ..., a_{3m}$, které značí velikosti prvků.
   (b) Takové kladné celé číslo B, že $\forall i \in \{1, 2, ..., 3m\} : \frac{B}{4} < a_i < \frac{B}{2}$ a $\sum_{i=1}^{3m} a_i = mB$, které značí velikost přihrádek.

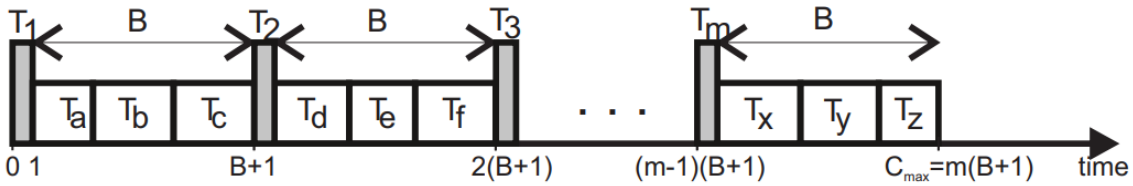
(c) Zkoumáme, jestli můžeme rozdělit A do m skupin $A_1, A_2, ..., A_m$ tak, že $\forall j \in \{1, 2, ..., m\} : \sum_{a_i \in A_j} a_i = B$.

2. Vyrobíme instanci I_{SCH} rozvrhovacího problému $1|r_j, \tilde{d_j}|C_{max}$ s $4m$ úlohami $T_j = (p_j, r_j, \tilde{d_j})$

   (a) $\forall i \in \{1, 2, ..., m\} : T_j = (1, (B+1) \cdot (j-1), (B+1) \cdot (j-1) + 1)$, toto jsou pomocné úkoly pro rozdělení přihrádek.

   (b) $\forall i \in \{m+1, ..., 4m\} : T_j = (a_i, 0, \infty), i = j - m$, každý z těchto úkolů T_j odpovídá prvku a_i v I_{3P}.

   (c) Pokud platí $C_{max} = m(B+1)$, pak odpověď na I_{3P} je ANO, jinak NE.



Úlohu můžeme reprezentovat pomocí ILP tak, že $x_{iq} = 1$ pokud úkol i je na q-té pozici v pořadí úkolů a t_q je start time úkolu na q-té pozici:

$$\min C_{max}$$

$$\sum_{q=1}^{n} x_{iq} = 1 \qquad \text{pro } i = 1...n \text{ - každý úkol má právě jednu pozici}$$

$$\sum_{i=1}^{n} x_{iq} = 1 \qquad \text{pro } q = 1...n \text{ - v každé pozici je právě jeden úkol}$$

$$t_q \geq \sum_{i=1}^{n} r_j \cdot x_{iq} \qquad \text{pro } q = 1...n \text{ - } s_j \text{ úkolu musí být nejdříve v jeho } r_j$$

$$t_q \geq t_{q-1} + \sum_{i=1}^{n} p_i \cdot x_{iq-1} \qquad \text{pro } q = 2...n \text{ - úkol může začít až po skončení předchozího}$$

$$t_q \leq \sum_{i=1}^{n} d_i \cdot x_{iq} - \sum_{i=1}^{n} p_i \cdot x_{iq} \qquad \text{pro } q = 1...n \text{ - úkol musí skončit před svojí deadline}$$

$$C_{max} \geq t_n + \sum_{i=1}^{n} p_i \cdot x_{in} \qquad \text{- } C_{max} \text{ musí být větší nebo roven skončení posledního úkolu}$$

6.2.5 Bratheyho algoritmus pro $1|r_j, \tilde{d}_j|C_{max}$

Algoritmus řeší úlohu $1|r_j, \tilde{d}_j|C_{max}$ pomocí Branch and Bound přístupu, tedy vytváří postupně všechna možná pořadí úkolů a zkouší, zda je (částečný) rozvrh validní. Algoritmus pracuje s asymptotickou složitostí $O(n!)$. Hlavní částí algoritmu jsou rozhodování pro ořazání jednotlivých větví, kdy v každém kroku provedeme tyto kontroly:

1. Pokud je deadline jakéhokoli nepřiřazeného úkolu dříve než aktuální C_{max}, ořezáváme tuto větev.

2. Pokud aktuální C_{max} částečného řešení je horší, než C_{max} již nalezeného řešení, ořezáváme tuto větev.

3. Pokud všechny nepřiřazené úkoly mají release time po aktuálním C_{max}, je toto částečné řešení optimální a víme jistě, že v jeho podstromu bude i celkové optimální řešení, můžeme tedy ořezat všechny ostatní větve.

Abychom ověřili, že aktuálně nalezené konečné řešení je optimální, využijeme BRTP (Block with Release Time Property) test optimality. BRTP je taková sada k úkolů, že úkol T_1 začíná ve svém release time, všechny úkoly jsou seřazeny těsně za sebe bez čekání (tedy nezáleží na přesném pořadí úkolů), a platí $r_1 \leq r_i$ pro $i = 2...k$. Pokud BRTP existuje, rozvrh je optimální, jelikož poslední úkol nemůže být proveden dříve a žádný úkol nemůže být proveden před r_1. Může se ale stát, že pro optimální řešení nebude existovat BRTP.

6.2.6 Branch and Bound s LP pro $1|prec|\sum w_j \cdot C_j$

Nejdříve formulujeme problém jako ILP:

1. Vytvoříme proměnné $x_{ij} \in \{0, 1\}$ tak, že $x_{ij} = 1$ právě tehdy když T_i je před T_j nebo $i = j$.

2. Zakódujeme precedence do proměnných $e_{ij} \in \{0, 1\}$ tak, že $e_{ij} = 1$ právě tehdy když existuje orientovaná hrana (T_i, T_j) v grafu precedence G.

3. Čas ukončení úkolu T_j se skládá z p_j a processing time jeho předchůdců:

$$C_j = \sum_{i=1}^{n} p_i \cdot x_{ij}$$
$$w_j \cdot C_j = \sum_{i=1}^{n} p_i \cdot x_{ij} \cdot w_j$$
$$J = \sum_{j=1}^{n} w_j \cdot C_j = \sum_{j=1}^{n} \sum_{i=1}^{n} p_i \cdot x_{ij} \cdot w_j$$
$$\min J(x) \text{ ze všech přípustných rozvrhů}$$

4. Celá ILP formulace tedy je:

$$\min \sum_{j=1}^{n} \sum_{i=1}^{n} p_i \cdot x_{ij} \cdot w_j$$

$$\begin{aligned}
x_{ij} &\geq e_{ij} && \text{pro } i,j \in 1...n \text{ - precedence v rozvrhu dodržuje graf } G \\
x_{ij} + x_{ji} &= 1 && \text{pro } i,j \in 1...n, i \neq j \text{ - buď } T_i \text{ je před } T_j \text{ nebo naopak} \\
1 \leq x_{ij} + x_{jk} + x_{ki} &\leq 2 && \text{pro } i,j,k \in 1...n, i \neq j \neq k \text{ - v grafu nejsou cykly} \\
x_{ii} &= 1 && \text{pro } i \in 1...n \text{ - každý úkol je v rozvrhu právě jednou}
\end{aligned}$$

Můžeme ho ale zvolnit a využít LP bounding tím, že $x_{ij} \in \mathbb{R}$ a $0 \leq x_{ij} \leq 1$. To nám sice nedá optimální řešení, ale můžeme vzít zbývající úkoly J^{LP} a použít je jako spodní hranici pro množství zbývající práce, tedy v následném Branch and Bound algoritmu budeme mít nové omezení pro ořezání větve: pokud J^{JP} + aktuální částečné řešení je horší než nalezené nejlepší řešení, ořezáváme větev.

6.2.7 EDD - Earliest Due Date first

Algoritmus seřadí všechny úkoly vzestupně podle jejich due date s asymptotickou složitostí $O(n \cdot log(n))$. Důkaz optimality je následující:

1. Mějme optimální rozvrh S^A vytvořený algoritmem A.
2. Mějme rozvrh S^{EDD} vytvořený algoritmem EDD.
3. Pokud $S^A \neq S^{EDD}$, pak existují dva úkoly T_a a T_b s $d_a \leq d_b$ takové, že T_b je v S^A těsně před T_a.

4. Prohození úkolů T_a a T_b nemůže prodloužit maximální zpoždění L_{max}, protože L_a a L_b se mohou prohozením pouze zkrátit nebo zůstat stejné.

5. Konečným počtem takových prohození změníme S^A na S^{EDD}.

6.2.8 Hornův algoritmus pro $1|pmtn, r_j|L_{max}$

Algoritmus řeší úlohu EDD v případě, že úkoly mají definované release time. Na vstupu je množina úkolů T s processing time, release time a due date. Na výstupu jsou start time pro každý úkol. Postup:

1. Najdeme minimální r_j pro všechny nerozvržené úkoly a zapíšeme ho do t_1.

2. Pokud mají všechny úkoly $r_j = t_1$, pak nastavíme $t_2 := \infty$, jinak bude t_2 minimální r_j větší než t_1.

3. Do množiny T' vložíme úkoly, které mají $r_j = t_1$ a odstraníme je z T.

4. Vybereme $T_k \in T'$ s minimálním d_j.

5. Rozvrhneme T_k na intervalu $\langle t_1, t_1 + min(p_k, t_2 - t_1)\rangle$.

6. Čekáme na jedno ze dvou přerušení

   (a) Pokud úkol T_k doběhl, vracíme se do bodu 4.

   (b) Pokud jsme došli do času t_2, vracíme se do bodu 3.

7. Pokud nejsou rozvržené všechny úkoly, vyprázdníme množinu T' a vracíme se do bodu 1.

6.2.9 Hornův algoritmus pro $1|pmtn, r_j, d_j = \tilde{d}_j|L_{max}$

Rozšíření Hornova algoritmu, které v průběhu rozvrhování kontroluje i deadline úkolů a pokud narazí na úkol, který překračuje svůj deadline, skonší neúspěšně. Nejedná se tedy pouze o optimalizační algoritmus, ale i o rozhodovací, protože nás informuje o proveditelnosti rozvrhu. Algoritmus řeší úlohu EDF (Earliest Deadline First). Důkaz optimality je velice podobný důkazu u EDD s tím rozdílem, že neprohazujeme celé úkoly, ale jenom jejich části, jelikož máme povolenou preempci.

6.2.10 Chetto, Silly, Bouchentouf algoritmus pro $1|pmtn, prec, r_j, d_j = \tilde{d}_j|L_{max}$

Algoritmus převádí množinu závislých úkolů T na množinu nezávislých úkolů T' (tedy zbaví se precedence) a následně řeší úlohu pomocí EDF. Postup:

1. Upravíme release time úkolů
   (a) Pro úkoly bez předků nastavíme $r'_j = r_j$.
   (b) Vybereme ty úkoly T_j, kterým nebyl nastaven r'_j, ale všem jejich předkům ano, a nastavíme jim $r'_j = max(r_j, max(r_h + p_h))$, kde T_h jsou přímí předci T_j.
   (c) Pokud nejsou upravené release time všech úkolů, vracíme se do bodu 2.
2. Upravíme deadline úkolů
   (a) Pro úkoly bez následníků nastavíme $\tilde{d}'_j = \tilde{d}_j$.
   (b) Vybereme ty úkoly T_j, kterým nebyl nastaven $\tilde{d}'_j$, ale všem jejich následníkům ano, a nastavíme jim $\tilde{d}'_j = min(\tilde{d}_j, min(\tilde{d}_h - p_h)$, kde T_h jsou přímí následníci T_j.
   (c) Pokud nejsou upravené deadline všech úkolů, vracíme se do bodu 2.
3. Na upravených hodnotách spustíme Hornův algoritmus.

6.3 Rozvrhování na paralelních identických zdrojích ($\alpha = P$)

Naším cílem je rozdělit úkoly na jednotlivé procesory tak, abychom minimalizovali celkovou dobu provádění úkolů a dodrželi časová omezení.

$P2\|\|C_{max}$	NP hard, můžeme redukovat z 2-Partition problému a porovnávat C_{max} s $\frac{1}{2} \cdot \sum_{i=1}^{n} p_i$
$P\|pmtn\|C_{max}$	triviální, řešíme pomocí McNaughtonova algoritmu
$P\|pmtn, r_j, \tilde{d}_j\|-$	triviální, rozhodovací verze problému maximálního toku
$P\|prec\|C_{max}$	NP hard, řešíme LS - aproximačním algoritmem
$P\|\|C_{max}$	NP hard, řešíme LPT - aproximačním algoritmem nebo Rothkopfovým algoritmem
$P\|pmtn, prec\|C_{max}$	NP hard, řešíme Muntz&Coffmanovým úrovňovým algoritmem

6.3.1 McNaughtonův algoritmus pro $P|pmtn|C_{max}$

Na vstupu máme množinu preemptivních úkolů T a číslo R značící počet procesorů. Na výstupu máme čtyři vektory s^1, s^2, z^1, z^2, kde s_i^1, s_i^2 obsahují start time první/druhé části úkolu T_i a z_i^1, z_i^2 obsahují index procesoru, na které je první/druhá část úkolu T_i. Algoritmus pracuje s asymptotickou složitostí $O(n)$. Myšlenkou algoritmu je přiřazovat úkoly na první volný procesor a případné přesahy přiřadit na další volný procesor. Postup:

1. Spočítáme $C_{max}^* := max(max_{i=1..n}(p_i), \frac{1}{R} \cdot \sum_{i=1}^{n} p_i)$, tedy optimální délku zpracování úkolů, která se rovná buď processing time nejdelšího úkolu a nebo sumě processing time všech úkolů vydělenou počtem procesorů.

2. Dokud dokončení posledního přiřazeného úkolu nepřesahuje C_{max}^*, přiřazujeme úkoly na první volný procesor.

3. Část úkolu, která přesahuje C_{max}^*, přesuneme na další volný procesor.

4. Pokud máme nepřiřazené úkoly, vrátíme se do bodu 2.

6.3.2 LS - aproximační algoritmus pro $P|prec|C_{max}$

Jedná se o aproximační algoritmus s faktorem $r = 2 - \frac{1}{R}$ pro náhodně seřazený list úkolů. Na vstupu máme množinu preemptivních úkolů T, číslo R značící počet procesorů a DAG reprezentující precendence úkolů. Na výstupu máme start time a resource ID pro každý úkol. Myšlenkou algoritmu je přiřazovat úkoly vždy na nejméně vytížený procesor. Postup:

1. Seřadíme úkoly do seznamu L, různé způsoby řazení vedou k různé hodnotě faktoru r.

2. Najdeme zdroj Z, který bude nejdříve uvolněný, tedy má zatím nejkratší procesní čas t_z.

3. Vytvoříme $S \subset L$ úkolů s nejmenší a_i availability time (rovná se t_k pokud úkol nemá předchůdce, ∞ pokud aspoň jeden předchůdce je v L, maximální $s_j + p_j$ svých předchůdců).

4. Z S vybereme úkol T_i, který je první v L a vložíme ho na zdroj Z s $s_j = max(t_z, a_i)$.

5. Pokud nemáme rozřazené všechny úkoly, vracíme se do bodu 2.

Jelikož se jedná a heuristický algoritmus, mohou se v něm vyskytnout zvláštní anomálie (C_{max} se zvýší, pokud uvolníme některé podmínky) způsobené snížením processing time, odstraněním precedenčních podmínek nebo zvýšením počtu zdrojů.

6.3.3 LPT - aproximační algoritmus pro $P||C_{max}$

Algoritmus pouze definuje řazení pro úkoly v LS algoritmu pomocí LPT (Longest Processing Time first), tedy seřadí úkoly sestupně podle p_j. Díky tomu se zlepší aproximační faktor na $r = \frac{4}{3} - \frac{1}{3R}$. Algoritmus pracuje s asymptotickou složitostí $O(n \cdot log(n))$ kvůli řazení.

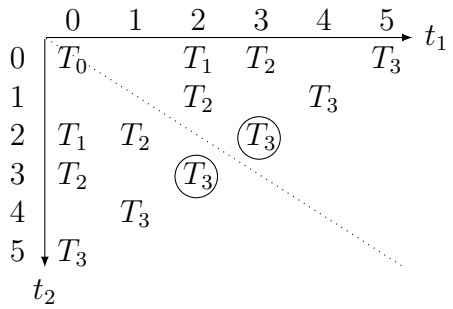
6.3.4 Rothkopfův algoritmus pro $P||C_{max}$

Tento pseudopolynomiální algoritmus je založený na dynamickém programování a pracuje s asymptotickou složitostí $O(n \cdot UB^2)$ pro konstantní R (slabě NP hard) nebo $O(n \cdot UB^R)$ pro polynomiálně omezené R (silně NP hard). Postup:

1. Vytvoříme proměnné $x_i(t_1, t_2, ..., t_R)$, kde $i = 1, 2, ..., n$ je index úkolu, $v = 1, 2, ..., R$ je index zdroje, $t_v = 0, 1, ..., UB$ je časová proměnná spojená se zdrojem v, UB je horní mez pro C_{max}.

2. Nastavíme $x_i(t_1, t_2, ..., t_R) = 1$ právě tehdy, když úkoly $T_1, T_2, ..., T_R$ mohou být přiřazené zdroji tak, že P_v je zaplněný v časovém intervalu $\langle 0, t_v \rangle; v = 1, 2, ..., R$.

$n = 3, R = 2, p = (2, 1, 2), UB = 5$



6.3.5 Muntz&Coffmanův úrovňový algoritmus pro $P|pmtn, prec|C_{max}$

Jedná se o aproximační algoritmus s faktorem $r = 2 - \frac{2}{R}$, který pracuje s asymptotickou složitostí $O(n^2)$. Postup:

1. Vytvoříme si list Z se všemi úkoly, které aktuálně můžeme zpracovávat (všichni jejich předchůdci už jsou zpracovaní).
2. Úkoly vybíráme z listu Z do listu S podle jejich úrovně; úroveň úkolu T_j je suma p_i (včetně p_j) nejdelší cesty z T_j do koncového úkolu v grafu precedence.
3. Pokud je více úkolů se stejnou úrovní, než je zdrojů, pak každý úkol dostane část kapacity $\beta := \frac{h}{|S|}$ (h je počet volných zdrojů); pokud máme méně úkolů na aktuální úrovni než zdrojů a v Z jsou nějaké úkoly, vracíme se do bodu 2 s nižší úrovní.
4. Algoritmus se posouvá v čase dokud některý úkol neskončí nebo dokud neklesne úroveň zpracovávaných úkolů na úroveň nezpracovávaného úkolu nebo dokud úroveň rychleji zpracovávaného úkolu neklesne na úroveň pomaleji zpracovávaného úkolu.
5. Pokud nejsou všechny úkoly zpracované, vracíme se do bodu 1.

6.4 Project Scheduling

$PS1\|temp\|C_{max}$	NP hard, máme jeden zdroj a temporální omezení úkolů
$PSm, 1\|temp\|C_{max}$	NP hard, máme m zdrojů o kapacitě 1, dané přiřazení úkolů ke zdrojům a temporální omezení úkolů

6.4.1 Temporální omezení

Máme sadu úkolů $T = (T_1, T_2, ..., T_n)$ reprezentovanou v orientovaném grafu G. Každý úkol má svůj processing time a hrana (T_i, T_j) s hodnotou l_{ij} znamená, že $s_j \geq s_i + l_{ij}$. V grafu G mohou být negativní hrany i negativní cykly, ale ne pozitivní cykly.

Pokud $l_{ij} = p_i$, jedná se o klasickou situaci precedence. Pokud $l_{ij} > p_i$, může úkol T_j začít až $l_{ij} - p_i$ časových jednotek po skončení úkolu T_i. Pokud $0 < l_{ij} < p_i$, jedná se o situaci, kdy úkol T_j nemusí čekat na dokončení úkolu T_i a umí např. pracovat s průběžnými výstupy úkolu T_i. Pokud $l_{ij} = 0$, musí úkol T_j začít nepozději ve stejnou chvíli jako úkol T_i. Pokud $l_{ij} < 0$, musí úkol T_i začít nejpozději l_{ij} časových jednotek po začátku T_j, jedná se tedy o relativní deadline startu T_i vůči startu T_j.

Pokud máme k dispozici jeden zdroj, je absence pozitivních cyklů **nutnou** podmínkou pro rozvrhnutelnost. Pokud máme k dispozici neomezené množství zdrojů, je absence pozitivních cyklů **nutnou a postačující** podmínkou rozvrhnutelnosti. Důvodem této podmínky je, že se obecně zabýváme hledáním nejdelších cest v grafu.

6.4.2 Časově indexovaný model pro $PS1|temp|C_{max}$

ILP model je založený na proměnné x_{it}, která se rovná 1 právě tehdy, když $s_i = t$. Předpokládáme, že processing time jsou kladná celá čísla. UB je horní mez C_{max}, tedy $UB = \sum_{i=1}^{n} max(p_i, max_{i,j=1...n}(l_{ij}))$. Start time úkolu T_i je $s_i = \sum_{t=0}^{UB-1} t \cdot x_{it}$. Výhodami tohoto modelu jsou: snadná rozšiřitelnost pro identické paralelní procesory a relativně málo omezení. Nevýhodou tohoto modelu je, že jeho velikost roste s UB.

$$\min C_{max}$$

$$\sum_{t=0}^{UB-1} (t \cdot x_{it}) + l_{ij} \leq \sum_{t=0}^{UB-1} (t \cdot x_{jt}) \quad \text{pro } l_{ij} \neq -\infty, i \neq j \text{ - temporální omezení}$$

$$\sum_{i=1}^{n} \sum_{k=\max(0,t-p_i+1)}^{t} x_{ik} \leq 1 \quad \text{pro } t = 0...UB-1 \text{ - omezení zdrojů}$$

$$\sum_{t=0}^{UB-1} x_{it} = 1 \quad \text{pro } i = 1...n \text{ - každý úkol je rozvržený právě jednou}$$

$$\sum_{t=0}^{UB-1} (t \cdot x_{it}) + p_{ij} \leq C_{max} \quad \text{pro } i = 1...n \text{ - spodní omezení pro } C_{max}$$

Podmínka omezení zdrojů zajišťuje, že žádné dva úkoly nebudou zpracovávány zároveň. Kdyby každá úloha trvala jednu časovou jednotku, stačilo by vzít sumu sloupce menší nebo rovnu 1, jelikož ale mohou úkoly trvat déle než jednu časovou jednotku, je potřeba se v rozvrhu dívat do minulosti, zda daný úkol stále neprobíhá. Představme si, že jsme v čase 5 a chceme zjistit, jestli můžeme rozvrhnout úkol T_2. Úkol T_1 trvá 3 časové jednotky, musíme se tedy ujistit, že v buňkách $x_{1,5}, x_{1,4}$ a $x_{1,3}$ není 1, protože to by znamenalo, že úkol T_1 v čase 5 stále probíhá. Pokud máme více úkolů, musíme stejnou kontrolu udělat u každého úkolu zvlášť pro každý časový okamžik, což nám zajišťuje podmínka na omezení zdrojů výše.

6.4.3 Relativně seřazený model pro $PS1|temp|C_{max}$

ILP model je založený na proměnné x_{ij}, která se rovná 1 právě tehdy, když úkol T_i je před úkolem T_j. Předpokládáme, že processing time jsou nezáporná reálná čísla. UB je horní mez C_{max}, tedy $UB = \sum_{i=1}^{n} max(p_i, max_{i,j=1...n}(l_{ij}))$. Start time úkolu T_i je s_i. Výhodou tohoto modelu je, že jeho velikost nezávisí na UB. Nevýhodou tohoto modelu je, že vyžaduje mnoho omezení.

$\min C_{max}$	
$s_i + l_{ij} \leq s_j$	pro $l_{ij} \neq -\infty, i \neq j$ - temporální omezení
$p_j \leq s_i - s_j + UB \cdot x_{ij} \leq UB - p_i$	pro $i, j = 1...n, i < j$ - omezení zdrojů
$s_i + p_i \leq C_{max}$	pro $i = 1...n$ - spodní omezení pro C_{max}

Podmínka omezení zdrojů zajišťuje pro každé dva úkoly, že budou mít mezi sebou dostatečný časový rozestup. UB je v tomto smyslu využívané jako "big M".

6.4.4 Relativně seřazený model pro $PSm, 1|temp|C_{max}$

ILP model je založený na proměnné x_{ij}, která se rovná 1 právě tehdy, když úkol T_i je před úkolem T_j. Proměnná m značí počet zdrojů a proměnné $a_1, a_2, ..., a_n$ značí přiřazení úkolů k jednotlivým zdrojům. Předpokládáme, že processing time jsou nezáporná reálná čísla. UB je horní mez C_{max}, tedy $UB = \sum_{i=1}^{n} max(p_i, max_{i,j=1...n}(l_{ij}))$. Start time úkolu T_i je s_i.

$\min C_{max}$	
$s_i + l_{ij} \leq s_j$	pro $l_{ij} \neq -\infty, i \neq j$ - temporální omezení
$p_j \leq s_i - s_j + UB \cdot x_{ij} \leq UB - p_i$	pro $i, j = 1...n, i < j, \underline{a_i = a_j}$ - omezení zdroje
$s_i + p_i \leq C_{max}$	pro $i = 1...n$ - spodní omezení pro C_{max}

Podmínka omezení na zdroji zajišťuje pro každé dva úkoly, které jsou rozvržené na stejném zdroji, že budou mít mezi sebou dostatečný časový rozestup. UB je v tomto smyslu využívané jako "big M".

6.4.5 Redukce z $1|r_j, \tilde{d}_j|C_{max}$ na $PS1|temp|C_{max}$

Tato polynomiální redukce dokazuje, že $PS1|temp|C_{max}$ je NP hard, protože Bratleyho problém je NP hard. Postup:

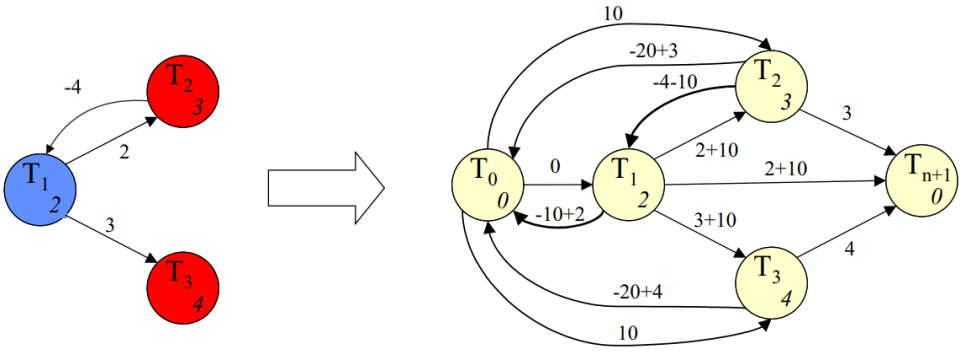
1. Vytvoříme nový úkol T_0 s $p_0 = 0$.
2. Vytvoříme hrany (T_0, T_i) pro $i = 1..n$ s $l_{0,i} = r_i$.
3. Vytvoříme hrany (T_i, T_0) pro $i = 1..n$ s $l_{i,0} = -(\tilde{d}_i - p_i)$.

6.4.6 Redukce z $PSm, 1|temp|C_{max}$ na $PS1|temp|C_{max}$

Tato redukce je založená na převodu každého zdroje do vlastního časového okna, tedy rozvrh úkolů na zdroji P^j je namapovaný do intervalu $\langle (j-1) \cdot UB, j \cdot UB \rangle$. Postup:

1. Vytvoříme pomocné úkoly T_0, T_{n+1} s $p_0 = p_{n+1} = 0$ tak, že úkol T_0 je zpracovávaný na zdroji P^1 jako první a úkol T_{n+1} je zpracovávaný na zdroji P^m jako poslední.
2. Přidáme nová temporální omezení $l_{0,i} = a_i \cdot UB$ a $l_{i,0} = -a_i \cdot UB + p_i$ pro $i = 1...n$, kterými zajistíme, že úkoly budou ve svých časových oknech.
3. Převedeme původní temporální omezení na $l'_{ij} = l_{ij} + (a_j - a_i) \cdot UB$, tedy k původnímu omezení přidáme jeho zařazení do správného časového okna.

Nové start time úkolů budou $s'_i = s_i + (a_i - 1) \cdot UB$.



6.4.7 Multiprocesorové úkoly

Převod z multiprocesorových úkolů na jednoprocesorové úkoly provedeme tak, že každý multiprocesorový úkol rozdělíme na tolik virtuálních úkolů, na kolika procesorech původní úkol bežel. Následně zajistíme, že tyto virtuální úkoly začnou ve stejný čas tak, že mezi nimi vytvoříme hrany s cenou 0.

7 Constraint programování

Příkladem constraint programování je hra sudoku. Máme hrací pole o velikosti 9x9, které máme vyplnit čísly 1 až 9, a určitá omezení (čísla v řádku/sloupci/čtverci se nesmějí opakovat, některá políčka jsou předvyplněná). Každé vyplnění políčka se iterativně propaguje do zbytku hracího pole. Může se stát, že existuje více řešení a nebo že řešení neexistuje.

7.1 Constraint satisfaction problém

Constraint satisfaction problém (CSP) je NP complete problém definovaný trojicí (X, D, C), kde $X = \{x_1, ..., x_n\}$ je konečná množina proměnných, $D = \{D_1, ..., D_n\}$ je konečná množina domén proměnných (doména je konečná množina přípustných hodnot pro danou proměnnou), $C = \{C_1, ..., C_n\}$ je konečná množina omezení (omezení je vazba $S_i \subseteq X$ a $R_i \subseteq D_{i_1} \times ... \times D_{i_n}$). Omezení většinou zapisujeme jako nerovnice, ale existují tzv. globální omezení, která obsahují složitější interní struktury a usnadňují nám následnou práci (např. rozvrhování, kliky a cykly v grafu, konečné automaty).

CSP je rozhodovací problém a hledá pouze první možné řešení, naopak CSOP (Constraint Satisfaction Optimization Problem) hledá nejoptimálnější řešení mezi všemi přípustnými (např. pomocí Branch and Bound algoritmu) podle kriteriální funkce $f(n)$. Constraint Solving je je definováno (X, D, C) kde $D_i \in \mathbb{R}$, Constraint programování obsahuje Constraint solving a Constraint satisfaction.

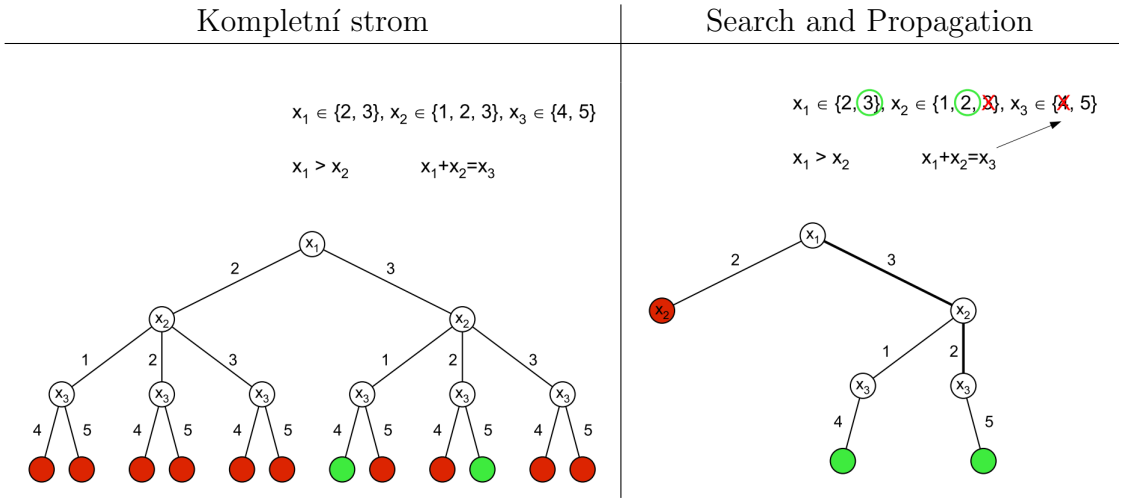
7.2 Search and Propagation

Search and Propagation je metoda na principu Branch and Bound, kdy rekurzivně procházíme všechny možné hodnoty proměnných a ořezáváme ty, které nejsou přípustné. Postup:

1. Nejprve ořežeme ty hodnoty proměnných, které nesplňují nějakou z našich podmínek s tím, že se nesmíme připravit o žádné možné řešení.

2. Vytvoříme strom možných řešení (z kořene vytvoříme potomky pro všechny možné hodnoty jedné proměnné a z těchto potomků další potomky pro hodnoty druhé proměnné atd.).

3. Projdeme všechny listy stromu a odřezáváme ty, které nesplňují naše podmínky, jakmile narazíme na první přípustné řešení, končíme.



7.3 Konzistence hran

Pokud budeme pracovat jen s binárním CSP, ve kterém každé omezení je jenom binární vztah. Binární CSP lze reprezentovat pomocí orientovaného grafu, kde uzly jsou jednotlivé proměnné (se svými přípustnými hodnotami) a hrany jsou omezení mezi proměnnými (hrany vedou oběma směry). Hrana (x_i, x_j) je konzistentní, pokud každou hodnotu x_i existuje hodnota x_j, která splňuje omezení na hraně. CSP je hranově konzistentní, pokud všechny hrany v grafu jsou konzistentní. Je důležité si pamatovat, že hrany jsou orientované a konzistence hrany (x_i, x_j) nezaručuje konzistenci hrany (x_j, x_i). Kromě globální konzistence nás může zajímat konzistence po cestě, k-konzistence, singleton konzistence atd. Revize hrany (x_i, x_j) je proces kontroly konzistence této hrany a pokud některé hodnoty v x_i nejsou konzistentní, jsou z x_i vyřazené.

7.4 AC-3 algoritmus

Algoritmus vychází z principu konzistence hran s tím, že si udržuje fronu hran k revizi (na začátku jsou ve frontě všechny hrany). Pokud při revizi hrany (x_i, x_j) dojde k vyřazení nějaké hodnoty z x_i, přidá algoritmus do fronty všechny hrany vcházející do x_i kromě hrany (x_j, x_i). Algoritmus končí, pokud mu dojdou hrany ve frontě, nebo pokud je doména nějaké proměnné prázdná.